

Aluno(a): Adilma Alves de Oliveira

1. Introdução

A verificação funcional é uma etapa fundamental no desenvolvimento de sistemas digitais, pois permite validar se um circuito implementa corretamente os requisitos especificados antes de sua síntese e fabricação. Por meio da simulação, é possível aplicar estímulos controlados, observar o comportamento dos sinais internos e externos do circuito e identificar possíveis falhas de projeto.

Nesta atividade foram analisados diferentes circuitos digitais descritos em Verilog HDL, juntamente com seus respectivos testbenches. Inicialmente, foram executadas as simulações utilizando os testbenches fornecidos, permitindo observar o funcionamento básico de cada circuito através das waveforms geradas. Em seguida, foram desenvolvidas versões aprimoradas dos testbenches utilizando recursos de SystemVerilog, incorporando mecanismos de self-checking para automatizar a comparação entre os resultados produzidos pelo circuito (DUT – Device Under Test) e os valores esperados calculados por modelos de referência.

Além da verificação funcional, foi utilizada uma infraestrutura de simulação baseada em Makefile, Icarus Verilog ([iverilog](#)), VVP e GTKWave, permitindo automatizar os processos de compilação, execução das simulações e análise das waveforms. Também foram realizadas análises temporais simples para avaliar o comportamento dos circuitos em relação ao clock e ao tempo de execução das simulações.

O objetivo deste relatório é apresentar a análise de cada circuito estudado, os estímulos aplicados durante os testes, os resultados obtidos, a interpretação das waveforms e as evidências que comprovam a correta implementação das funcionalidades especificadas.

2. Infraestrutura de Simulação com Makefile

Para automatizar o processo de compilação, simulação e análise dos circuitos, foi utilizado um Makefile. Essa ferramenta permite executar sequências de comandos através de regras pré-definidas, simplificando o fluxo de trabalho e reduzindo a possibilidade de erros durante a execução dos testes.

O fluxo utilizado neste laboratório é composto pelas seguintes etapas:

Compilação

A compilação é realizada pelo comando:

```
iverilog -g2012 -o sim/arquivo.out rtl/circuito.v tb/testbench.sv
```

Onde:

- **iverilog** é o compilador Verilog/SystemVerilog;
- **-g2012** habilita recursos do padrão SystemVerilog 2012;
- **-o sim/arquivo.out** define o nome do executável gerado;
- **rtl/circuito.v** corresponde ao arquivo RTL do circuito que será verificado;
- **tb/testbench.sv** corresponde ao testbench responsável por gerar os estímulos e verificar o comportamento do circuito.

Por exemplo:

```
iverilog -g2012 -o sim/mux_self.out rtl/8x1mux.v tb/tb_8x1mux_selfcheck.sv
```

Nesse caso, o compilador combina o circuito **8x1mux.v** com o testbench **tb_8x1mux_selfcheck.sv**, gerando o executável **mux_self.out**.

Execução da Simulação

Após a compilação, a simulação é executada pelo comando:

```
vvp sim/mux_self.out
```

O programa **vvp** executa o arquivo compilado e processa os estímulos definidos no testbench. Durante essa etapa são exibidas mensagens de PASS/FAIL e é gerado o arquivo de waveform no formato VCD.

Geração das Waveforms

Os arquivos VCD são criados através dos comandos:

```
$dumpfile("sim/mux8x1_selfcheck.vcd");  
$dumpvars(0, mux8x1_tb);
```

O comando `$dumpfile` define o nome do arquivo VCD que será gerado, enquanto `$dumpvars` registra as alterações dos sinais durante a simulação.

Visualização das Waveforms

As waveforms são abertas utilizando o GTKWave:

```
gtkwave sim/mux8x1_selfcheck.vcd
```

O GTKWave permite visualizar graficamente os sinais do circuito, facilitando a análise temporal e funcional do comportamento observado.

Limpeza dos Arquivos Gerados

Para remover arquivos temporários produzidos durante as simulações, utiliza-se a regra:

```
make clean
```

Essa regra executa:

```
rm -f sim/*.out sim/*.vcd
```

removendo os executáveis gerados pelo compilador e os arquivos de waveform.

Dessa forma, o Makefile padroniza o ambiente de verificação, permitindo compilar, simular e analisar diferentes circuitos através de comandos simples e reutilizáveis.

3. Circuito 1 - Demux 1x8

3.1. Descrição do Circuito

O circuito analisado implementa um demultiplexador 1x8 (DEMUX 1:8). Sua função é direcionar o valor presente na entrada de dados (**din**) para uma das oito posições da saída (**dout[7:0]**), de acordo com o valor aplicado ao sinal de seleção (**sel[2:0]**).

O módulo possui os sinais de entrada **clk**, **rstn**, **en**, **sel** e **din**. O sinal **rstn** corresponde a um reset assíncrono ativo em nível baixo, responsável por zerar a saída quando acionado. O sinal **en** atua como habilitação do circuito, permitindo a atualização da saída apenas quando está em nível lógico alto.

Embora a função principal seja a de um demultiplexador, a implementação utiliza um bloco sensível à borda de subida do clock (**posedge clk**), caracterizando um comportamento sequencial. Dessa forma, a saída é atualizada somente nas bordas de subida do clock quando o circuito está habilitado.

Quando **din = 1**, apenas o bit da saída correspondente ao valor de **sel** assume nível lógico alto, permanecendo os demais bits em zero. Quando **din = 0**, todas as saídas permanecem em zero.

3.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo (**rstn = 0**) com o objetivo de inicializar o circuito e garantir que a saída **dout** assumisse o valor zero. Após a liberação do reset (**rstn = 1**), o sinal de habilitação (**en**) foi ativado para permitir a atualização da saída.

No testbench original, foram aplicadas sequencialmente todas as oito combinações possíveis do sinal de seleção (**sel**), variando de **000** a **111**, mantendo a entrada de dados (**din**) em nível lógico alto. O objetivo foi verificar se o valor da entrada era corretamente direcionado para a posição correspondente da saída.

Na versão com self-checking, os testes foram ampliados para contemplar dois cenários distintos:

- **din = 0** e **sel** variando de **000** a **111**;
- **din = 1** e **sel** variando de **000** a **111**.

Dessa forma, foram executadas 16 transações ao todo. Para cada combinação testada, um modelo de referência calculou automaticamente o resultado esperado (**expected**), que foi comparado com a saída produzida pelo DUT (**dout**).

Esses estímulos foram escolhidos para validar completamente o comportamento funcional do demultiplexador, cobrindo todas as combinações possíveis do sinal de seleção e os dois estados possíveis da entrada de dados.

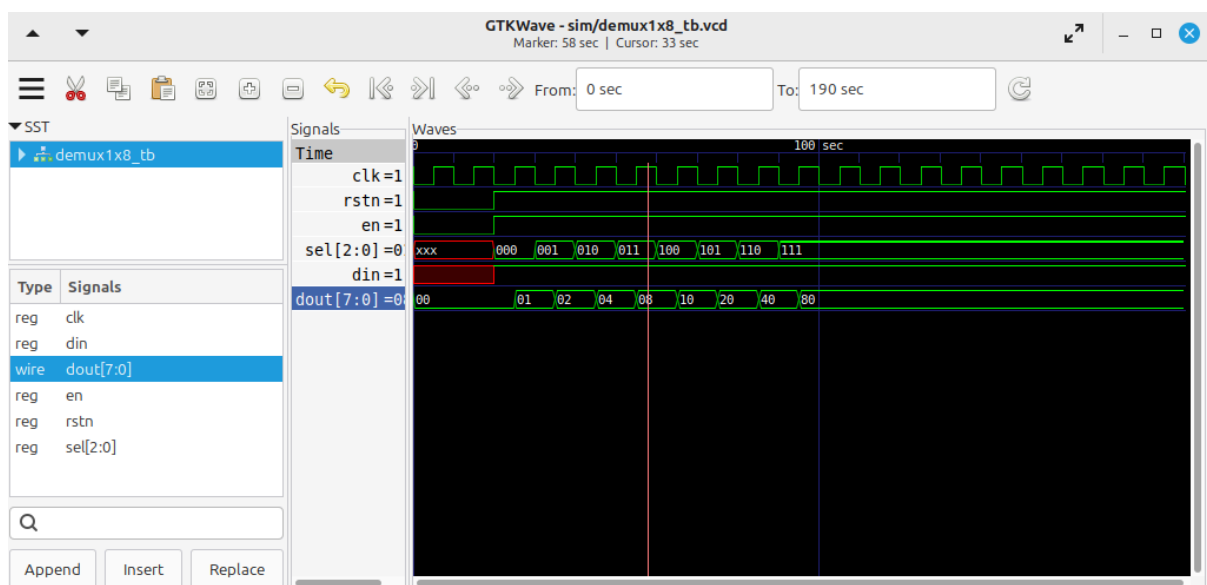
3.3. Resultados Obtidos

3.3.1. Testbench Original

A simulação inicial foi realizada utilizando o testbench fornecido pelo professor. A análise da waveform mostrou que, após a liberação do reset e a ativação do sinal de habilitação, o valor presente na entrada foi corretamente direcionado para a posição da saída indicada pelo sinal de seleção (**sel**).

Observou-se que, para cada valor de **sel**, apenas o bit correspondente em **dout** assumiu valor lógico alto, enquanto os demais permaneceram em zero. Os resultados observados foram compatíveis com o comportamento esperado de um demultiplexador 1x8.

Também foi observada a presença temporária do valor indefinido (**X**) no sinal **sel** durante o início da simulação, antes da aplicação dos estímulos. Após a inicialização dos sinais, o comportamento tornou-se determinístico e compatível com o esperado.



3.3.2. Testbench com Self-Checking

Foi implementado um mecanismo de self-checking utilizando um modelo de referência (**expected**) para calcular automaticamente o resultado esperado para cada combinação de entrada.

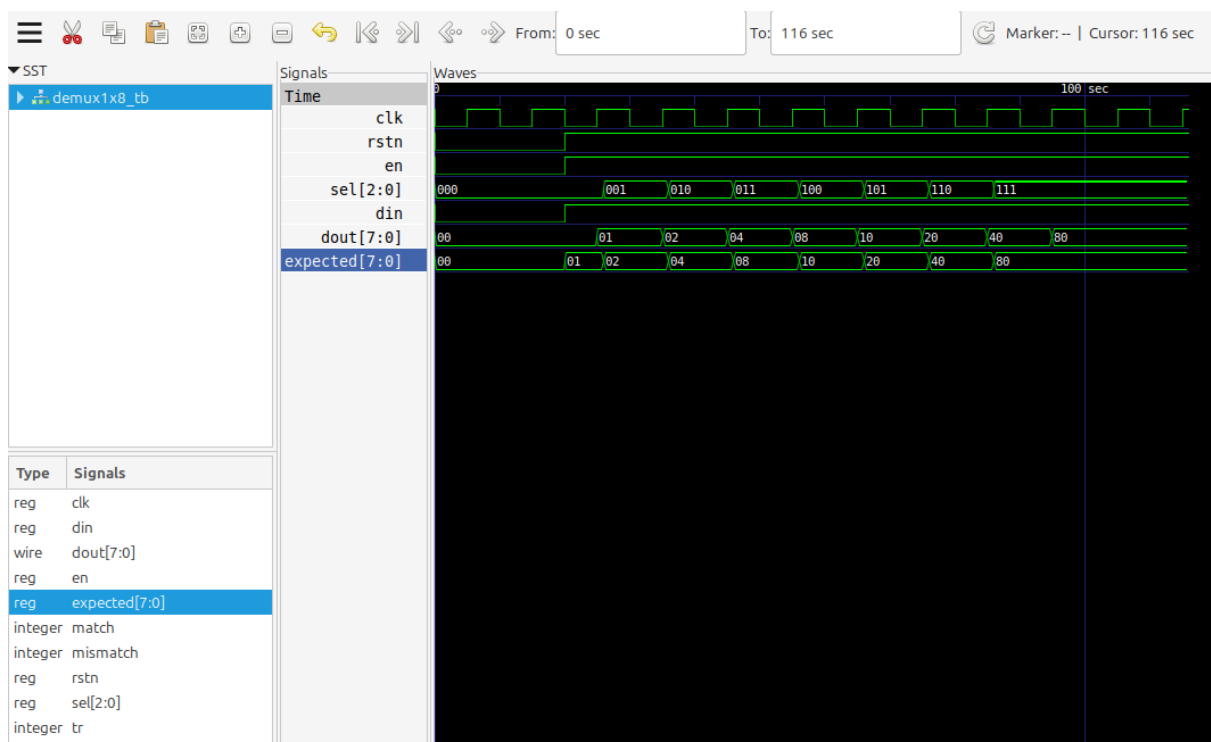
Foram executadas 16 transações, correspondentes a todas as combinações do sinal de seleção (**sel**) para os casos **din = 0** e **din = 1**.

Durante a simulação, os valores produzidos pelo DUT (**dout**) foram comparados automaticamente com os valores calculados pelo modelo de referência (**expected**). Em todas as verificações realizadas, os valores coincidiram.

O relatório final da simulação apresentou:

- Total de transações: 16
- Testes aprovados (PASS): 16
- Testes reprovados (FAIL): 0

A waveform confirmou que os sinais **dout** e **expected** permaneceram idênticos durante toda a execução dos testes, evidenciando a correta



implementação do circuito.

3.4. Análise das Waveforms

No início da simulação foi observada a presença do valor indefinido ("X") no sinal de seleção (**sel**). Esse comportamento ocorre porque o sinal ainda não havia recebido um valor válido durante a fase inicial da simulação. Após a aplicação dos estímulos pelo testbench, o sinal passou a assumir valores definidos, permitindo a correta avaliação do comportamento do circuito.

A waveform obtida durante a simulação do demultiplexador 1x8 com mecanismo de self-checking. Após a liberação do reset (**rstn=1**) e habilitação do circuito (**en=1**), foram aplicados os valores de seleção de **000** até **111**, mantendo a entrada **din=1**.

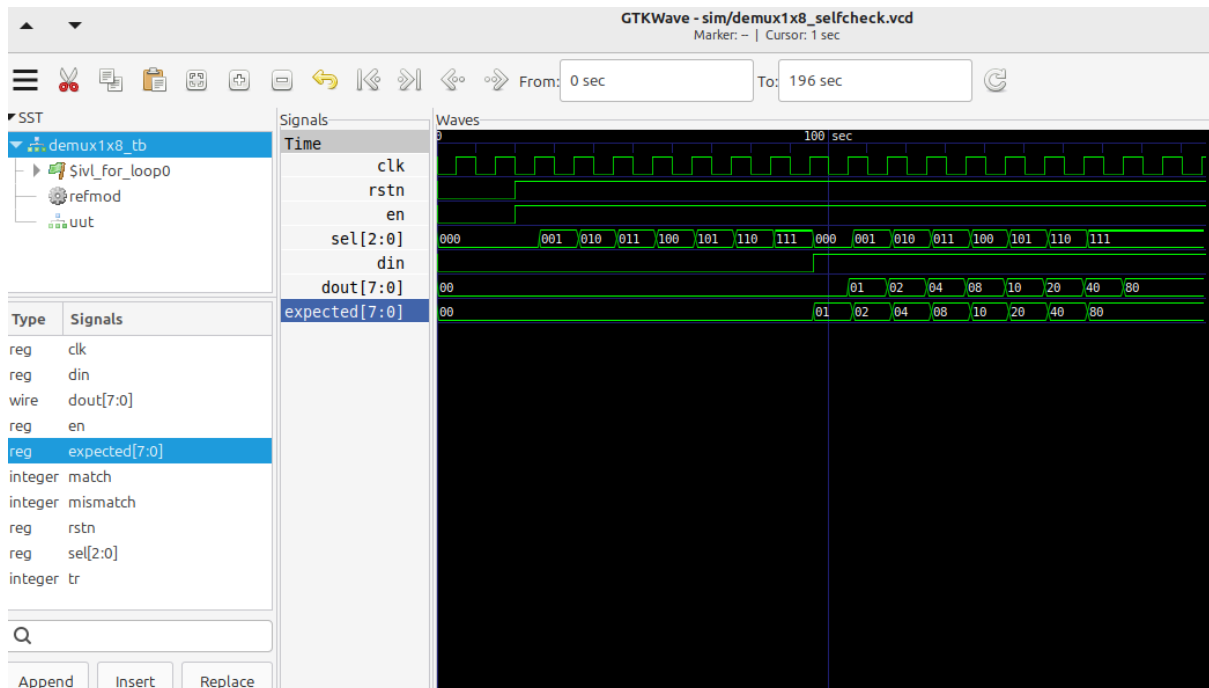
O sinal **expected** representa o resultado calculado pelo modelo de referência implementado no testbench. Observa-se que os sinais **expected** e **dout** permanecem idênticos durante toda a simulação, indicando que o DUT produziu exatamente os resultados esperados para todas as combinações testadas.

Para **sel=000**, a saída foi **00000001**; para **sel=001**, a saída foi **00000010**; para **sel=010**, a saída foi **00000100**, e assim sucessivamente até **sel=111**, cuja saída foi **10000000**. Dessa forma, foi possível validar o correto funcionamento do demultiplexador.

Para aumentar a cobertura funcional do teste, foram avaliados dois cenários distintos para a entrada de dados (**din**): valor lógico 0 e valor lógico 1. Para cada cenário, foram aplicadas todas as oito combinações possíveis do sinal de seleção (**sel**), totalizando 16 transações.

Quando **din=0**, observou-se que todas as saídas permaneceram em zero independentemente do valor de **sel**, conforme esperado. Quando **din=1**, verificou-se que apenas o bit correspondente ao valor de **sel** assumiu valor lógico alto, permanecendo os demais bits em zero.

O mecanismo de self-checking comparou automaticamente a saída do DUT (**dout**) com o valor calculado pelo modelo de referência (**expected**). Em todas as transações realizadas, os valores obtidos coincidiram com os valores esperados, resultando em 16 verificações bem-sucedidas e nenhuma falha.



3.5. Conclusão

O circuito demultiplexador 1x8 apresentou comportamento compatível com sua especificação funcional. Os testes realizados, incluindo todas as combinações de seleção e os dois estados possíveis da entrada de dados, foram aprovados sem falhas. A implementação do mecanismo de self-checking permitiu validar automaticamente os resultados, aumentando a confiabilidade da verificação.

4. Circuito 2 - 8x1 Mux

4.1. Descrição do Circuito

O circuito analisado implementa um multiplexador 8x1 (MUX 8:1). Sua função é selecionar um dos oito bits presentes na entrada de dados (`din[7:0]`) e encaminhá-lo para a saída (`dout`), de acordo com o valor aplicado ao sinal de seleção (`sel[2:0]`).

O módulo possui os sinais de entrada `clk`, `rstn`, `en`, `din` e `sel`. O sinal `rstn` corresponde a um reset assíncrono ativo em nível baixo, utilizado para inicializar a saída do circuito. O sinal `en` atua como habilitação, permitindo a atualização da saída apenas quando estiver em nível lógico alto.

A implementação utiliza um bloco sensível à borda de subida do clock (`posedge clk`), caracterizando um circuito sequencial. Dessa forma, as alterações na saída não ocorrem imediatamente após a mudança das entradas, mas apenas na próxima borda positiva do clock.

O funcionamento do multiplexador consiste em selecionar um dos oito bits da entrada `din` com base no valor de `sel`, transferindo esse valor para a saída `dout`.

4.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo (`rstn = 0`) com o objetivo de inicializar o circuito e garantir que a saída assumisse um estado conhecido. Após a liberação do reset (`rstn = 1`), o sinal de habilitação (`en`) foi ativado para permitir o funcionamento do multiplexador.

No testbench original, foram aplicados padrões de entrada contendo apenas um bit ativo por vez. Simultaneamente, o sinal de seleção (`sel`) percorreu todas as combinações possíveis de `000` a `111`. O objetivo foi verificar se o circuito selecionava corretamente o bit correspondente da entrada.

Na versão com self-checking, os testes foram ampliados para incluir dois grupos de estímulos:

- Padrões contendo um único bit ativo na entrada (`00000001`, `00000010`, `00000100`, ..., `10000000`);
- Padrão totalmente zerado (`00000000`).

Para cada combinação aplicada, o modelo de referência calculou automaticamente o valor esperado da saída (`expected`), que foi posteriormente comparado com o valor produzido pelo DUT (`dout`).

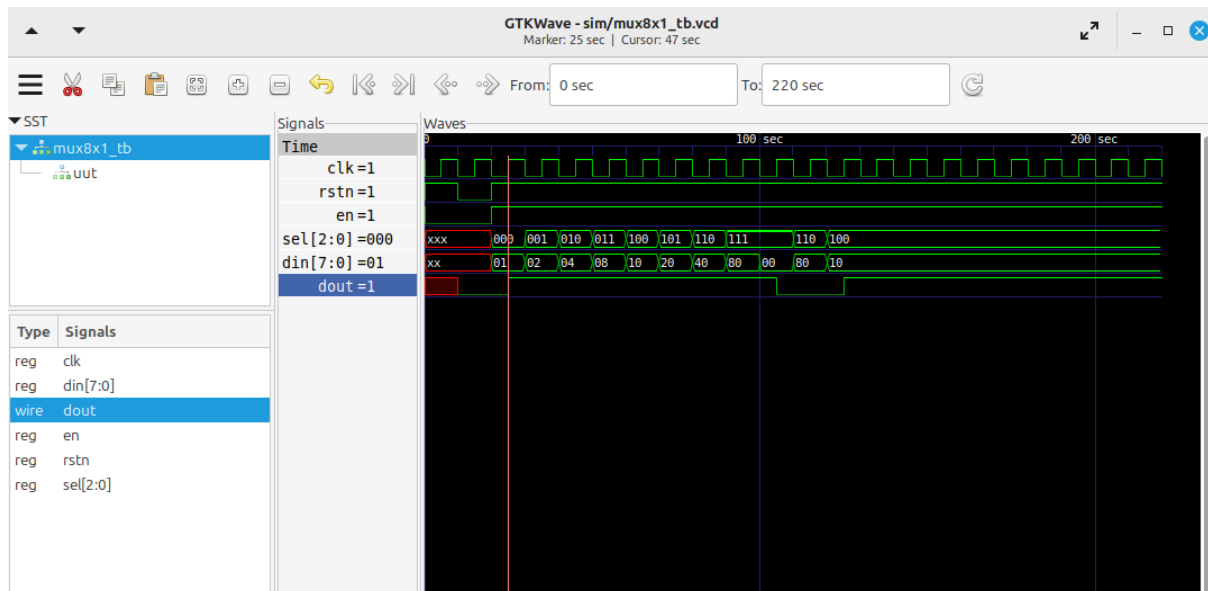
Ao todo, foram executadas 16 transações, cobrindo todas as combinações do sinal de seleção para os dois cenários de entrada.

4.3. Resultados Obtidos

4.3.1. Testbench Original

No início da simulação foram observados valores indefinidos (X) nos sinais `sel`, `din` e `dout`. Isso ocorre porque esses sinais ainda não haviam sido inicializados pelo testbench. Após a aplicação dos estímulos e a liberação do reset, os sinais passam a assumir valores definidos.

Observa-se um atraso entre a ativação de `en` e a atualização de `dout`, pois o circuito utiliza lógica sequencial sensível à borda de subida do clock. Assim, a saída não muda imediatamente após a alteração de `en`, mas apenas na borda positiva seguinte de `clk`.



4.3.2. Testbench com Self-Checking

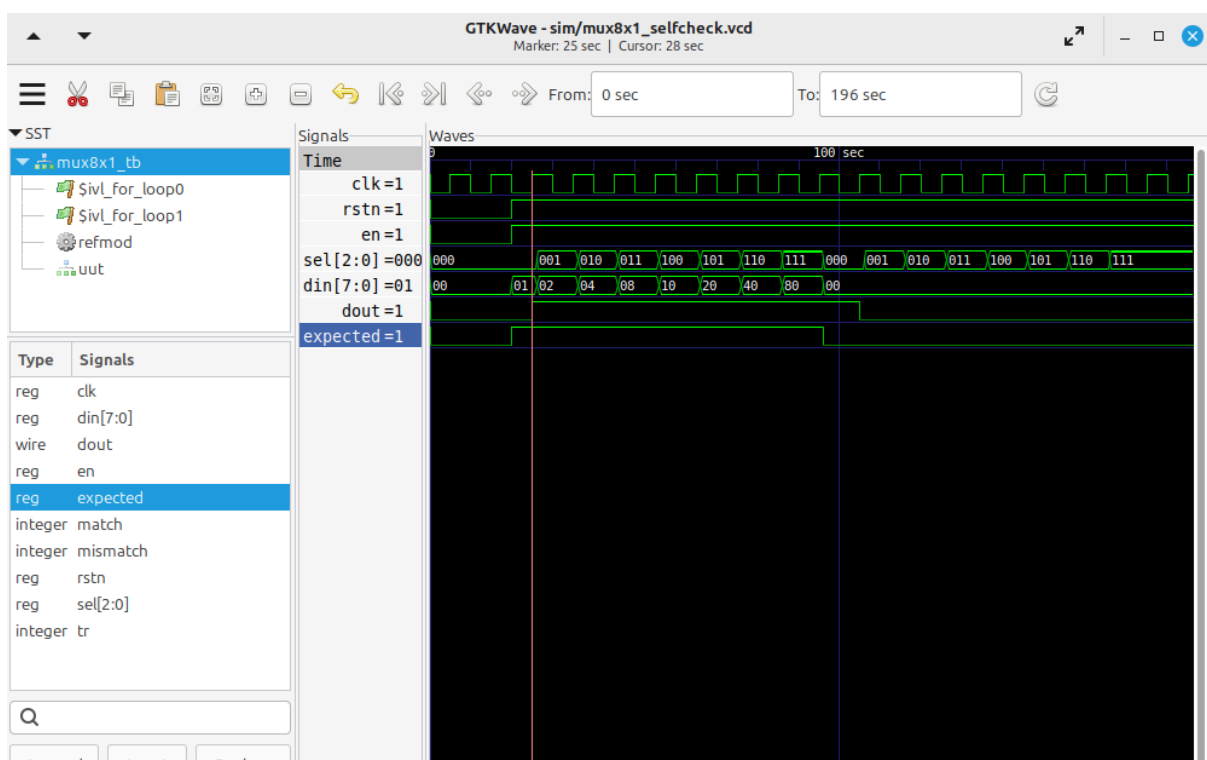
Foi implementado um mecanismo de self-checking utilizando um modelo de referência (`expected`) para calcular automaticamente o valor esperado da saída do multiplexador.

Foram executadas 16 transações, divididas em dois grupos de testes. No primeiro grupo, foram aplicados padrões de entrada contendo apenas um bit ativo, enquanto o sinal de seleção percorreu todas as combinações possíveis. Nessa

condição, a saída permaneceu em nível lógico alto, pois o bit selecionado em cada teste possuía valor igual a 1.

No segundo grupo, foi aplicado o valor `00000000` na entrada `din`, mantendo a variação do sinal de seleção. Como todos os bits da entrada estavam em nível lógico baixo, a saída permaneceu em zero para todas as combinações testadas.

Durante toda a simulação, os valores produzidos pelo DUT (`dout`) coincidiram com os valores calculados pelo modelo de referência (`expected`). O relatório final apresentou 16 transações aprovadas e nenhuma falha, validando o funcionamento do multiplexador 8x1.



4.4. Análise das Waveforms

A waveform obtida a partir do testbench original demonstra o funcionamento básico do multiplexador. Após a inicialização do circuito, o sinal de seleção (`sel`) percorre todas as combinações possíveis, enquanto diferentes padrões são aplicados à entrada `din`. Observa-se que a saída `dout` assume o valor do bit selecionado em cada instante de tempo.

No início da simulação foram observados valores indefinidos (X) nos sinais `sel`, `din` e `dout`. Esse comportamento é esperado e ocorre porque os sinais ainda não haviam sido inicializados pelo testbench. Após a aplicação dos estímulos, todos os sinais passaram a apresentar valores definidos.

Também é possível observar um atraso entre a ativação do sinal de habilitação (`en`) e a atualização da saída (`dout`). Esse comportamento ocorre porque o circuito é sequencial e atualiza a saída apenas na borda de subida do clock.

Na waveform gerada pelo testbench com self-checking, foi adicionado o sinal `expected`, correspondente ao valor calculado pelo modelo de referência. Observa-se que os sinais `dout` e `expected` permanecem coincidentes durante toda a simulação, evidenciando que o DUT produziu exatamente os resultados esperados para todos os casos testados.

Durante a primeira etapa dos testes, foram aplicados padrões contendo um único bit ativo, resultando em `dout = 1` para todas as seleções. Na segunda etapa, com a entrada totalmente zerada, a saída permaneceu em nível lógico baixo independentemente do valor de `sel`. A concordância entre `dout` e `expected` confirma a correta implementação funcional do multiplexador 8x1.

4.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do multiplexador 8x1. Foram verificadas todas as combinações possíveis do sinal de seleção (`sel`), utilizando diferentes padrões de entrada. A implementação do mecanismo de self-checking permitiu comparar automaticamente a saída do DUT com os resultados calculados pelo modelo de referência (`expected`), resultando em 16 verificações aprovadas e nenhuma falha.

As waveforms obtidas corroboram os resultados apresentados pelo scoreboard, evidenciando que o circuito atende à sua especificação funcional. Em todas as transações executadas, os valores produzidos pelo DUT coincidiram com os valores esperados calculados pelo modelo de referência.

Análise Temporal

Além da verificação funcional, foi realizada uma medição do tempo total de execução da simulação utilizando a função `$time` do SystemVerilog. O tempo foi registrado no início e ao final da sequência de testes, resultando em um tempo total de execução de 176 unidades de simulação.

Os resultados mostraram que todas as 16 transações foram executadas dentro do intervalo temporal previsto pelo testbench. A análise também confirmou que a atualização da saída ocorre apenas após a borda ativa do clock, comportamento esperado para um circuito sequencial síncrono. Dessa forma, tanto os aspectos funcionais quanto os aspectos temporais do multiplexador foram validados com sucesso.

5. Circuito 3 - Decoder 2x4

5.1. Descrição do Circuito

O circuito analisado implementa um decoder 2x4, cuja função é converter uma entrada binária de 2 bits (`din[1:0]`) em uma saída de 4 bits (`dout[3:0]`). O processo de decodificação consiste em ativar apenas uma das posições da saída para cada combinação possível da entrada.

O módulo possui os sinais de entrada `clk`, `rstn`, `en` e `din`. O sinal `rstn` corresponde a um reset assíncrono ativo em nível baixo, responsável por inicializar a saída do circuito. O sinal `en` atua como habilitação, permitindo que a saída seja atualizada somente quando estiver em nível lógico alto.

Embora a função de decodificação seja tipicamente associada a circuitos combinacionais, a implementação utilizada neste projeto emprega um bloco sensível à borda de subida do clock (`posedge clk`). Dessa forma, a saída é atualizada apenas na ocorrência de uma borda positiva do clock quando o circuito está habilitado.

Quando a entrada assume o valor `00`, a saída correspondente é `0001`. Para a entrada `01`, a saída torna-se `0010`. Quando a entrada é `10`, a saída passa para `0100`. Finalmente, para a entrada `11`, a saída assume o valor `1000`. Assim, cada combinação da entrada ativa exclusivamente uma posição da saída, caracterizando o comportamento esperado de um decoder 2x4.

5.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo ($\text{rstn} = 0$) com o objetivo de inicializar o circuito e garantir que a saída assumisse um estado conhecido antes do início dos testes. Após a liberação do reset ($\text{rstn} = 1$), o sinal de habilitação (en) foi ativado para permitir o funcionamento do decoder.

Foram aplicadas sequencialmente todas as combinações possíveis da entrada $\text{din}[1:0]$, correspondentes aos valores 00, 01, 10 e 11. O objetivo desses estímulos foi verificar se o circuito realizava corretamente a decodificação de cada valor de entrada, ativando apenas o bit correspondente na saída $\text{dout}[3:0]$.

Para cada estímulo aplicado, o modelo de referência (refmod) calculou automaticamente o valor esperado da saída (expected). Em seguida, o valor produzido pelo circuito sob teste (DUT) foi comparado com o valor esperado através do mecanismo de self-checking. Essa abordagem permitiu validar automaticamente o comportamento do circuito sem depender exclusivamente da inspeção visual das waveforms.

Além da verificação funcional, foi implementada uma medição temporal utilizando a função $\$time$ do SystemVerilog. O instante inicial da simulação foi registrado antes da aplicação dos estímulos e o instante final foi registrado após a conclusão dos testes. Com essas informações foi possível calcular o tempo total de execução da simulação, bem como o tempo médio gasto por transação.

Ao todo foram aplicados 4 estímulos, cobrindo todas as combinações possíveis da entrada do decoder, garantindo a validação completa do comportamento funcional do circuito.

5.3. Resultados Obtidos

5.3.1. Testbench Original

A waveform obtida a partir da simulação do decoder 2x4 demonstra o comportamento esperado do circuito após a aplicação dos estímulos definidos no testbench.

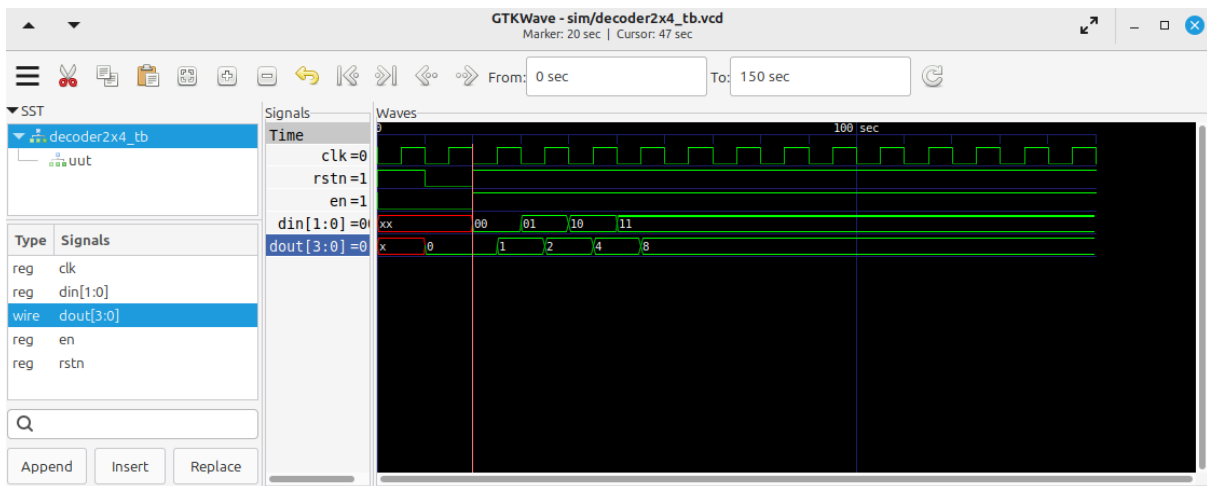
No início da simulação foram observados valores indefinidos (X) nos sinais din e dout , representados em vermelho pelo GTKWave. Esse comportamento ocorre porque, durante os instantes iniciais da simulação, os sinais ainda não haviam recebido valores válidos pelo testbench e o circuito ainda não havia sido inicializado.

completamente. Após a aplicação do reset e dos primeiros estímulos, os sinais passaram a assumir valores definidos, eliminando as regiões indefinidas.

Também foi observado que as alterações no sinal de entrada **din** ocorrem próximas às bordas de descida do clock. Isso acontece devido à forma como os estímulos foram programados no testbench, utilizando atrasos temporais fixos (**#10**). Como o clock possui período de 10 unidades de tempo, as mudanças de entrada acabam coincidindo com as bordas de descida do sinal de clock.

Entretanto, a atualização da saída **dout** ocorre apenas nas bordas de subida do clock. Esse comportamento é esperado, pois o circuito foi implementado utilizando um bloco sequencial sensível à borda positiva do clock (**posedge clk**). Dessa forma, existe um atraso entre a alteração da entrada e a atualização da saída, correspondente ao tempo necessário para a ocorrência da próxima borda ativa do clock.

Observa-se ainda que, para cada valor aplicado em **din**, a saída assume corretamente o valor correspondente ao processo de decodificação: **00** → **0001**, **01** → **0010**, **10** → **0100** e **11** → **1000**, confirmando o correto funcionamento do decoder 2x4.



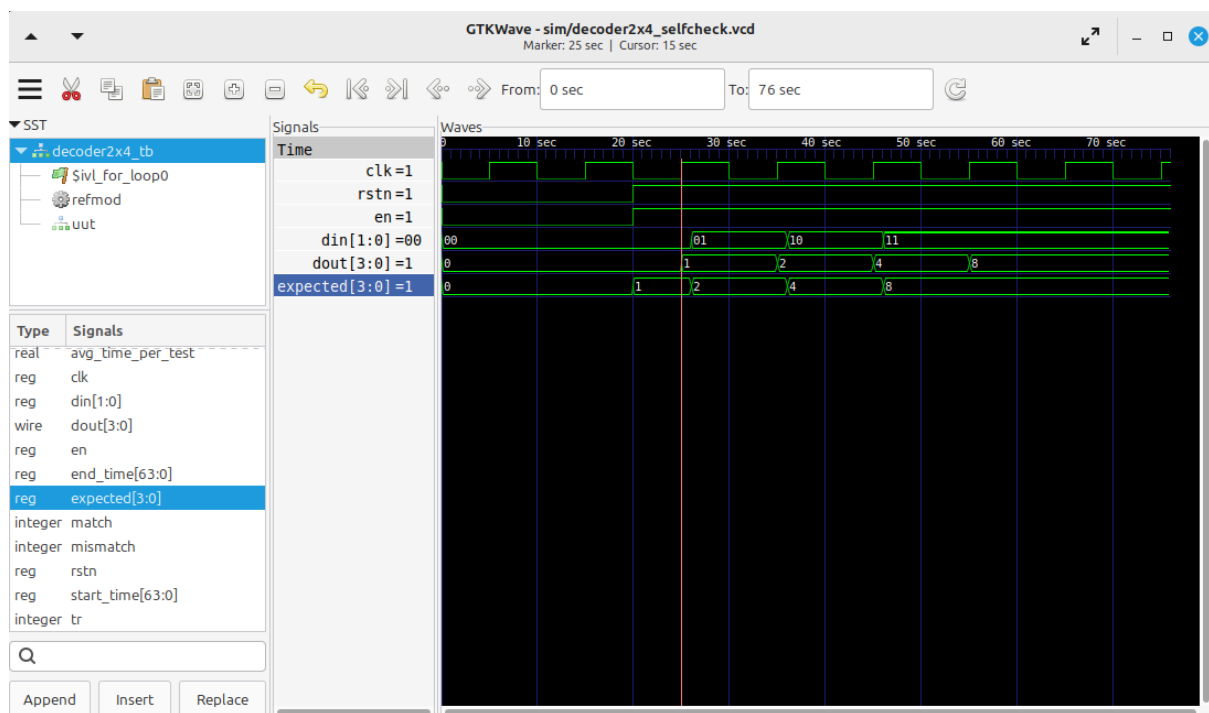
5.3.2. Testbench com Self-checking

Na versão com self-checking foi adicionado o sinal **expected**, responsável por armazenar o resultado calculado pelo modelo de referência. A comparação entre **expected** e **dout** permite verificar automaticamente o comportamento do circuito durante a simulação.

Observa-se que os sinais `expected[3:0]` e `dout[3:0]` permanecem coincidentes durante toda a execução dos testes, indicando que o circuito produziu exatamente os resultados esperados para todas as combinações da entrada `din`.

Também é possível observar que a saída não é atualizada imediatamente após a alteração da entrada. A atualização ocorre apenas na borda de subida do clock, comportamento compatível com a implementação do circuito, que utiliza um bloco sequencial sensível a `posedge clk`.

Os resultados apresentados na waveform confirmam os valores esperados do processo de decodificação: `00 → 0001`, `01 → 0010`, `10 → 0100` e `11 → 1000`. A concordância entre o DUT e o modelo de referência comprova o correto funcionamento do decoder 2x4.



5.4. Análise das Waveforms

A waveform obtida a partir do testbench com self-checking demonstra o correto funcionamento do decoder 2x4 para todas as combinações possíveis da entrada `din[1:0]`.

Inicialmente, o circuito é mantido em reset, garantindo que a saída assuma um estado conhecido antes da aplicação dos estímulos. Após a liberação do reset e

a ativação do sinal de habilitação (**en**), foram aplicadas sequencialmente as quatro combinações possíveis da entrada: **00**, **01**, **10** e **11**.

Observa-se que o sinal **expected[3:0]**, calculado pelo modelo de referência, acompanha exatamente o comportamento do sinal **dout[3:0]**, produzido pelo DUT (Device Under Test). Essa coincidência entre os dois sinais durante toda a simulação evidencia que o circuito implementa corretamente a função de decodificação.

Também é possível observar que a saída é atualizada apenas após a borda de subida do clock. Esse comportamento está de acordo com a implementação do circuito, que utiliza um bloco sequencial sensível ao evento **posedge clk**. Dessa forma, existe um pequeno atraso entre a alteração da entrada **din** e a atualização da saída **dout**, correspondente ao tempo necessário para a ocorrência da próxima borda ativa do clock.

Os valores observados na saída correspondem exatamente aos resultados esperados para um decoder 2x4: quando **din = 00**, a saída assume o valor **0001**; para **din = 01**, a saída torna-se **0010**; para **din = 10**, a saída passa para **0100**; e para **din = 11**, a saída assume o valor **1000**.

Além da validação funcional, a simulação registrou um total de 4 estímulos aplicados, todos aprovados pelo mecanismo de self-checking, sem ocorrência de falhas. O tempo total de execução foi de 56 unidades de simulação, resultando em um tempo médio de 14 unidades por transação.

5.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do decoder 2x4. Foram verificadas todas as combinações possíveis da entrada **din[1:0]**, permitindo validar o processo de decodificação para cada valor aplicado.

A implementação do mecanismo de self-checking possibilitou a comparação automática entre a saída produzida pelo DUT (**dout**) e os valores calculados pelo modelo de referência (**expected**). Durante a simulação foram aplicados 4 estímulos, correspondentes a todas as combinações possíveis da entrada, resultando em 4 verificações aprovadas e nenhuma falha.

A análise das waveforms confirmou que a saída do circuito segue o comportamento esperado, apresentando os valores **0001**, **0010**, **0100** e **1000** para

as respectivas entradas 00, 01, 10 e 11. Também foi possível observar que a atualização da saída ocorre apenas na borda de subida do clock, comportamento compatível com a implementação sequencial do circuito.

Além da validação funcional, foi realizada uma análise temporal da simulação. O tempo total de execução foi de 56 unidades de simulação, resultando em um tempo médio de 14 unidades por transação. Dessa forma, os resultados obtidos demonstram que o decoder 2x4 atende corretamente às especificações propostas e que tanto seu comportamento funcional quanto temporal foram validados com sucesso.

6. Circuito 4 - Encoder 4x2

6.1. Descrição do Circuito

O circuito analisado implementa um encoder 4x2, cuja função é converter uma entrada de 4 bits (`din[3:0]`) em uma saída codificada de 2 bits (`dout[1:0]`). O processo de codificação consiste em identificar qual bit da entrada está ativo e representar essa posição por meio de um código binário de 2 bits na saída.

O módulo possui os sinais de entrada `clk`, `rstn`, `en` e `din`. O sinal `rstn` corresponde a um reset assíncrono ativo em nível baixo, responsável por inicializar a saída do circuito. O sinal `en` atua como habilitação, permitindo a atualização da saída apenas quando estiver em nível lógico alto.

Embora a função de codificação seja tipicamente associada a circuitos combinacionais, a implementação utilizada neste projeto emprega um bloco sensível à borda de subida do clock (`posedge clk`). Dessa forma, a saída é atualizada somente na ocorrência de uma borda positiva do clock quando o circuito está habilitado.

Quando a entrada assume o valor 0001, a saída correspondente é 00. Para a entrada 0010, a saída torna-se 01. Quando a entrada é 0100, a saída passa para 10. Finalmente, para a entrada 1000, a saída assume o valor 11. Dessa forma, o circuito realiza a codificação da posição do bit ativo na entrada, produzindo uma representação binária equivalente na saída.

6.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo (`rstn = 0`) com o objetivo de inicializar o circuito e garantir que a saída assumisse um estado conhecido antes do início dos testes. Após a liberação do reset (`rstn = 1`), o sinal de habilitação (`en`) foi ativado para permitir o funcionamento do encoder.

Foram aplicados sequencialmente os quatro padrões válidos de entrada utilizados pelo circuito: `0001`, `0010`, `0100` e `1000`. Esses estímulos foram escolhidos porque representam todas as combinações previstas pelo encoder para identificar a posição do bit ativo na entrada e convertê-la para um código binário de 2 bits na saída.

Para cada estímulo aplicado, o modelo de referência (`refmod`) calculou automaticamente o valor esperado da saída (`expected`). Em seguida, o mecanismo de self-checking comparou esse valor com a saída produzida pelo DUT (`dout`), permitindo validar automaticamente o comportamento do circuito.

Além da verificação funcional, foi implementada uma medição temporal utilizando a função `$time` do SystemVerilog. O instante inicial foi registrado antes da execução dos testes e o instante final após a conclusão da sequência de estímulos. Com isso, foi possível determinar o tempo total de execução da simulação e o tempo médio gasto por transação.

Ao todo foram aplicados 4 estímulos, cobrindo todas as entradas válidas previstas para o encoder 4x2.

6.3. Resultados Obtidos

6.3.1. Testbench Original

A simulação do encoder 4x2 foi executada utilizando o testbench fornecido juntamente com o circuito. Após a aplicação do reset e a ativação do sinal de habilitação (`en`), foram aplicados sequencialmente os padrões de entrada `0001`, `0010`, `0100` e `1000`, correspondentes às quatro combinações válidas para o processo de codificação.

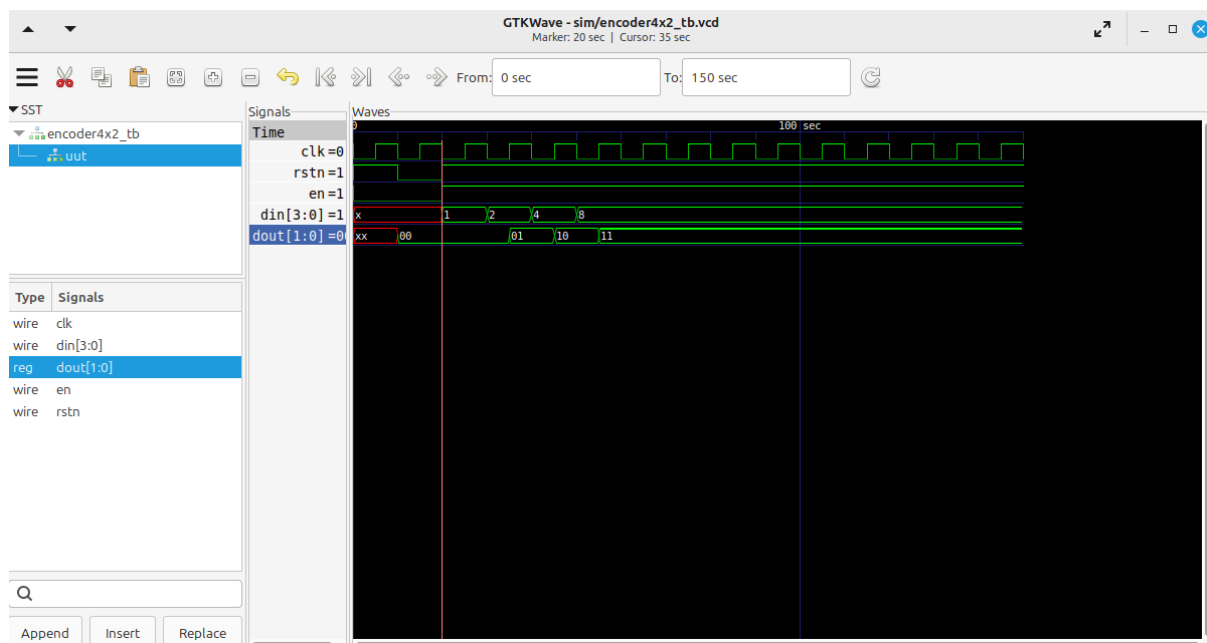
A análise da waveform mostrou que a saída `dout[1:0]` respondeu corretamente a cada estímulo aplicado. Para a entrada `0001`, a saída assumiu o

valor 00; para 0010, a saída tornou-se 01; para 0100, a saída passou para 10; e para 1000, a saída assumiu o valor 11.

No início da simulação foram observados valores indefinidos (X) nos sinais `din` e `dout`, representados em vermelho pelo GTKWave. Esse comportamento é esperado, pois os sinais ainda não haviam sido inicializados e o circuito não havia recebido estímulos válidos. Após a aplicação do reset e dos primeiros valores de entrada, os sinais passaram a apresentar estados definidos.

Também foi possível observar que a saída é atualizada apenas após a borda de subida do clock, evidenciando o comportamento sequencial do circuito. Dessa forma, existe um pequeno atraso entre a alteração da entrada e a atualização da saída, correspondente ao tempo necessário para a ocorrência da próxima borda ativa do clock.

Os resultados observados são compatíveis com a funcionalidade esperada de um encoder 4x2, indicando que o circuito realiza corretamente a codificação da posição do bit ativo da entrada em uma representação binária de 2 bits na saída.



6.3.2. Testbench com Self-checking

Na versão com self-checking foi adicionado um modelo de referência (`expected`) responsável por calcular automaticamente o valor esperado da saída para cada estímulo aplicado. Esse valor foi comparado com a saída produzida pelo

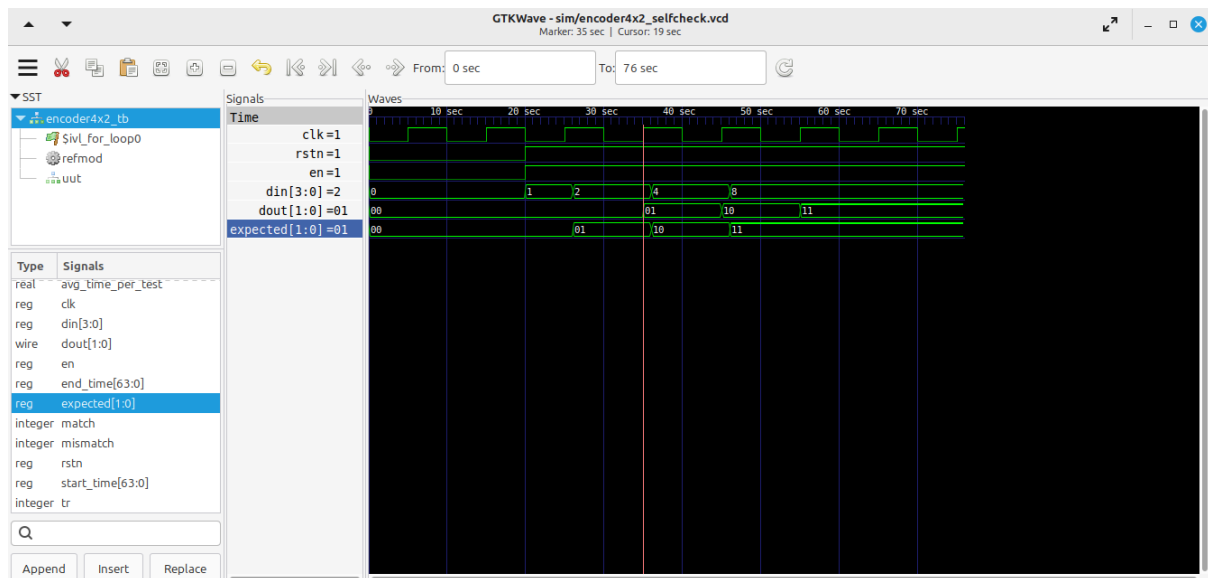
DUT (**dout**), permitindo a validação automática do comportamento do encoder durante toda a simulação.

Observa-se na waveform que os sinais **expected[1:0]** e **dout[1:0]** permanecem coincidentes durante toda a execução dos testes, indicando que o circuito produziu exatamente os resultados previstos pelo modelo de referência. Essa concordância é confirmada pelo relatório final da simulação, que registrou 4 estímulos aplicados, 4 verificações aprovadas e nenhuma falha.

Também é possível observar um atraso entre a aplicação de um novo valor em **din** e a atualização correspondente da saída **dout**. Esse comportamento ocorre porque o circuito foi implementado utilizando um bloco sequencial sensível à borda de subida do clock (**posedge clk**). Dessa forma, mesmo que a entrada seja alterada, a saída somente é atualizada na próxima borda positiva do clock. Esse atraso é esperado e confirma o funcionamento síncrono do circuito.

Durante os testes foram aplicadas as quatro combinações válidas da entrada (**0001**, **0010**, **0100** e **1000**). Para cada uma delas, a saída assumiu corretamente os valores codificados **00**, **01**, **10** e **11**, respectivamente. A análise da waveform demonstra que o encoder executa corretamente o processo de codificação e que o mecanismo de self-checking foi capaz de validar automaticamente todos os resultados obtidos.

Além da verificação funcional, foi realizada uma análise temporal utilizando a função **\$time** do SystemVerilog. O tempo total de execução da simulação foi de 56 unidades de simulação, resultando em um tempo médio de 14 unidades por estímulo aplicado. Esses resultados indicam que a sequência de testes foi executada conforme planejado e que o comportamento temporal do circuito está de acordo com sua implementação síncrona.



6.4. Análise das Waveforms

A waveform obtida a partir do testbench com self-checking demonstra o correto funcionamento do encoder 4x2 para todas as combinações válidas de entrada testadas.

Observa-se que os sinais `expected[1:0]` e `dout[1:0]` permanecem coincidentes durante toda a simulação, indicando que os resultados produzidos pelo circuito correspondem exatamente aos valores calculados pelo modelo de referência. Essa concordância confirma a eficácia do mecanismo de self-checking implementado.

Também é possível observar um atraso entre a aplicação de um novo valor em `din` e a atualização correspondente da saída `dout`. Esse comportamento ocorre porque a saída é atualizada apenas na borda de subida do clock, conforme definido pelo bloco sequencial sensível a `posedge clk`. Dessa forma, as alterações da entrada somente são refletidas na saída após a ocorrência da próxima borda ativa do clock.

Durante a simulação, os valores aplicados à entrada foram codificados corretamente. Para `din = 0001`, a saída assumiu o valor `00`; para `0010`, a saída tornou-se `01`; para `0100`, a saída passou para `10`; e para `1000`, a saída assumiu o valor `11`. A correspondência entre os sinais `expected` e `dout` ao longo de toda a waveform evidencia que o encoder executa corretamente a sua função de codificação.

6.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do encoder 4x2. Foram verificadas todas as entradas válidas previstas para o circuito, permitindo validar o processo de codificação para cada padrão aplicado.

A implementação do mecanismo de self-checking possibilitou a comparação automática entre a saída produzida pelo DUT (**dout**) e os valores calculados pelo modelo de referência (**expected**). Durante a simulação foram aplicados 4 estímulos, resultando em 4 verificações aprovadas e nenhuma falha.

A análise das waveforms confirmou que a saída do circuito segue o comportamento esperado, produzindo os códigos binários correspondentes à posição do bit ativo na entrada. Também foi possível observar que a atualização da saída ocorre apenas na borda de subida do clock, comportamento compatível com a implementação sequencial do circuito.

Além da validação funcional, foi realizada uma análise temporal da simulação. O tempo total de execução foi de 56 unidades de simulação, resultando em um tempo médio de 14 unidades por transação. Dessa forma, os resultados obtidos demonstram que o encoder 4x2 atende corretamente às especificações propostas e que tanto seu comportamento funcional quanto temporal foram validados com sucesso.

7. Circuito 5 - Arbiter

7.1. Descrição do Circuito

O circuito analisado implementa um árbitro (arbiter) responsável por controlar o acesso de dois clientes a um recurso compartilhado. Seu objetivo é decidir qual cliente receberá permissão de acesso quando uma ou mais requisições forem realizadas simultaneamente.

O módulo possui como entradas os sinais **client1_req** e **client2_req**, que representam as solicitações dos dois clientes, além do sinal **priority_sel**, utilizado para definir qual cliente possui prioridade em situações de disputa. O circuito também utiliza os sinais **clk** e **reset_n** para sincronização e inicialização da máquina de estados.

As saídas do módulo são **o_grant1** e **o_grant2**, responsáveis por indicar qual cliente recebeu autorização para utilizar o recurso compartilhado. Em qualquer

instante, apenas uma das saídas pode permanecer ativa, garantindo exclusão mútua no acesso ao recurso.

A implementação é baseada em uma máquina de estados finitos (FSM), composta por estados que representam a ausência de concessão e as situações em que o acesso foi concedido ao cliente 1 ou ao cliente 2. As transições entre estados são determinadas pelas requisições recebidas e pela política de prioridade definida pelo sinal `priority_sel`, permitindo que o circuito realize a arbitragem de forma controlada e previsível.

7.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo (`reset_n = 0`) com o objetivo de inicializar a máquina de estados do árbitro em uma condição conhecida. Após a liberação do reset, foram aplicadas diferentes combinações dos sinais `client1_req`, `client2_req` e `priority_sel`, permitindo avaliar o comportamento do circuito em situações de ausência de requisições, requisições individuais e requisições simultâneas.

Foram executados oito cenários distintos, contemplando todas as combinações relevantes de solicitações dos clientes e das configurações de prioridade. Para cada estímulo aplicado, o modelo de referência calculou automaticamente os valores esperados para os sinais de concessão (`expected_grant1` e `expected_grant2`), que foram comparados com as saídas produzidas pelo circuito sob teste (`o_grant1` e `o_grant2`).

Além da validação funcional, foi realizada uma medição temporal utilizando a função `$time` do SystemVerilog. O instante inicial foi registrado antes da aplicação dos estímulos e o instante final após a conclusão da sequência de testes. Com isso foi possível determinar o tempo total de execução da simulação e o tempo médio por transação. Ao final foram aplicados 8 estímulos, cobrindo todos os cenários previstos para a arbitragem entre os dois clientes.

7.3. Resultados Obtidos

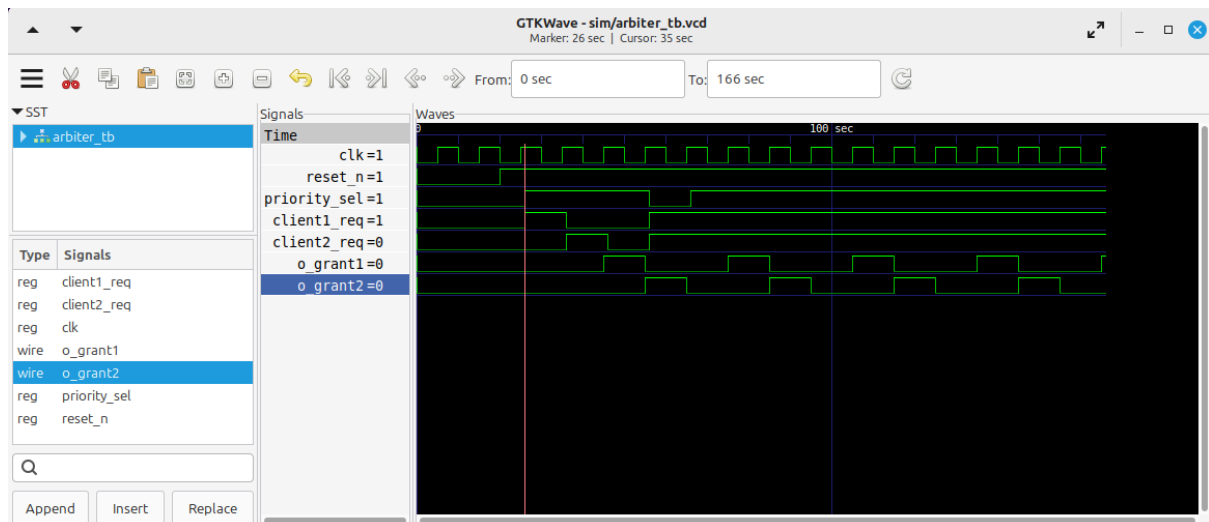
7.3.1. Testbench Original

A simulação do árbitro foi executada utilizando o testbench fornecido juntamente com o circuito. Após a aplicação do reset, foram realizados diferentes cenários de requisição pelos clientes, permitindo verificar o mecanismo de arbitragem implementado pelo módulo.

Inicialmente, foi testada uma situação em que apenas o cliente 1 realizava uma requisição, resultando na ativação do sinal `o_grant1`. Em seguida, foi aplicado um cenário em que apenas o cliente 2 solicitava acesso ao recurso compartilhado, ocasionando a ativação do sinal `o_grant2`.

Posteriormente, foram avaliadas situações em que ambos os clientes realizavam requisições simultaneamente. Nesses casos, o comportamento observado dependeu do valor do sinal `priority_sel`. Quando a prioridade foi atribuída ao cliente 2, o sinal `o_grant2` permaneceu ativo. Quando a prioridade foi alterada para o cliente 1, o circuito passou a conceder acesso ao cliente 1 através do sinal `o_grant1`.

Os resultados observados demonstram que o circuito executa corretamente a função de arbitragem, concedendo acesso ao cliente apropriado de acordo com as requisições recebidas e com a política de prioridade definida pelo sinal de seleção.



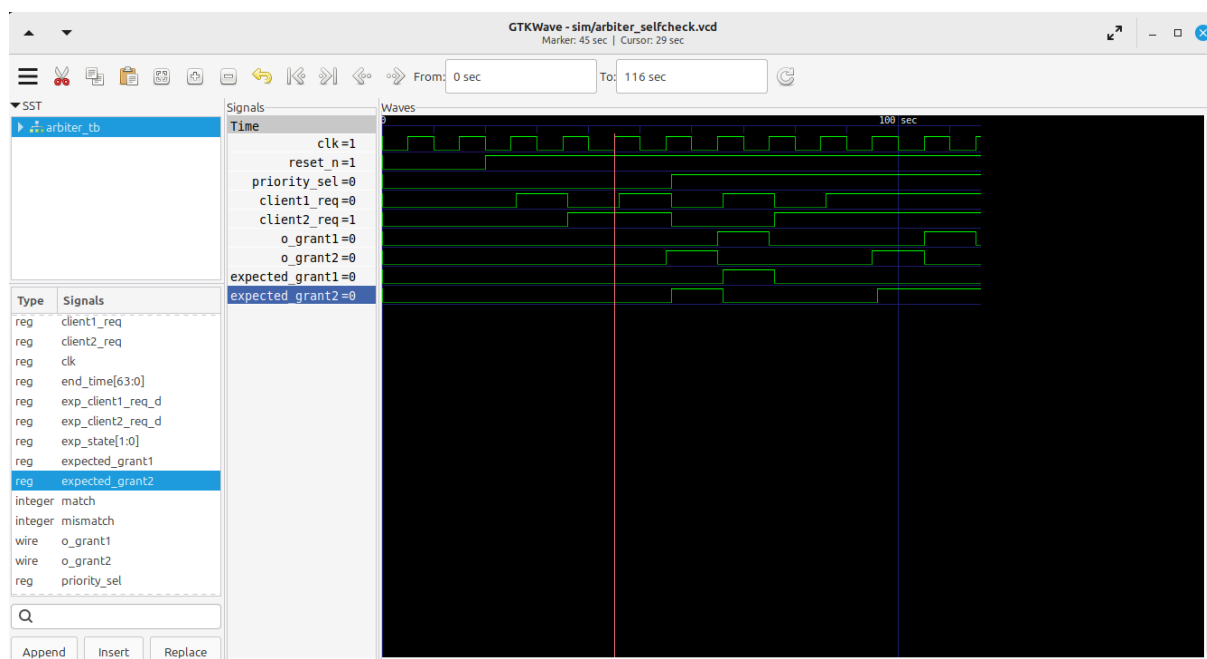
7.3.2. Testbench com Self-checking

O testbench foi modificado para incorporar um mecanismo de self-checking capaz de verificar automaticamente o comportamento do árbitro durante a simulação. Para isso, foi implementado um modelo de referência responsável por reproduzir a lógica de arbitragem e calcular os valores esperados para os sinais de concessão (**expected_grant1** e **expected_grant2**).

Diferentemente dos circuitos anteriores, o árbitro é baseado em uma máquina de estados finitos (FSM), exigindo que o modelo de referência considerasse não apenas os valores atuais das entradas, mas também o estado interno da arbitragem. Dessa forma, o modelo foi desenvolvido para acompanhar as transições de estado e reproduzir o comportamento observado no circuito sob teste.

Durante a simulação, cada conjunto de estímulos foi aplicado ao DUT e ao modelo de referência. Após a atualização dos estados internos sincronizada pelo clock, os sinais de saída produzidos pelo circuito (**o_grant1** e **o_grant2**) foram comparados com os sinais esperados calculados pelo modelo (**expected_grant1** e **expected_grant2**). Sempre que os valores coincidiam, uma mensagem de aprovação (PASS) era registrada; caso contrário, era contabilizada uma falha (FAIL).

Ao final da execução foram aplicados 8 estímulos, resultando em 8 verificações aprovadas e nenhuma falha. O mecanismo de self-checking permitiu validar automaticamente o funcionamento da máquina de estados do árbitro, reduzindo a necessidade de inspeção manual das waveforms e aumentando a confiabilidade do processo de verificação.



7.4. Análise das Waveforms

A waveform obtida a partir do testbench com self-checking demonstra o correto funcionamento do árbitro para todos os cenários de teste aplicados. Os sinais `expected_grant1` e `expected_grant2`, calculados pelo modelo de referência, permanecem coincidentes com os sinais `o_grant1` e `o_grant2` produzidos pelo DUT durante toda a simulação.

Observa-se que as saídas não são atualizadas imediatamente após a alteração das entradas. As mudanças ocorrem apenas após a ocorrência da borda ativa do clock, comportamento esperado para um circuito sequencial baseado em máquina de estados. Dessa forma, existe um pequeno atraso entre a aplicação dos estímulos e a atualização dos sinais de concessão.

Também é possível verificar que apenas um dos sinais de grant permanece ativo por vez, garantindo acesso exclusivo ao recurso compartilhado. Nos cenários em que ambos os clientes realizam solicitações simultaneamente, a decisão tomada pelo circuito respeita corretamente a configuração do sinal `priority_sel`, evidenciando o correto funcionamento do mecanismo de arbitragem.

A concordância entre os sinais esperados e observados ao longo de toda a waveform confirma que a máquina de estados implementada no árbitro executa corretamente as transições previstas e concede acesso ao cliente adequado em cada situação analisada.

7.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do árbitro implementado. Foram avaliados diferentes cenários de requisição envolvendo os dois clientes, incluindo situações de acesso individual e de concorrência simultânea ao recurso compartilhado.

A implementação do mecanismo de self-checking permitiu comparar automaticamente as saídas produzidas pelo DUT com os valores calculados pelo modelo de referência. Durante a simulação foram aplicados 8 estímulos, resultando em 8 verificações aprovadas e nenhuma falha.

A análise das waveforms demonstrou que os sinais de concessão gerados pelo circuito coincidem integralmente com os sinais esperados calculados pelo

modelo de referência. Também foi possível verificar que as decisões de arbitragem são realizadas de forma sincronizada com o clock e que a política de prioridade definida pelo sinal `priority_sel` é respeitada em todos os cenários avaliados.

A análise temporal registrou um tempo total de execução de 96 unidades de simulação, resultando em um tempo médio de 12 unidades por transação. Dessa forma, os resultados obtidos demonstram que o árbitro atende corretamente às especificações funcionais e temporais propostas, validando o funcionamento da máquina de estados responsável pelo controle de acesso ao recurso compartilhado.

8. Circuito 6 - Counter 32 bit

8.1. Descrição do Circuito

O circuito analisado implementa um contador síncrono de 32 bits com detecção de overflow. Seu funcionamento é controlado pelos sinais `clk`, `reset_n`, `en` e `load`. Quando o sinal de habilitação (`en`) está ativo, o contador incrementa seu valor a cada borda de subida do clock. O sinal `reset_n`, ativo em nível baixo, força a reinicialização do contador para zero, enquanto o sinal `load` permite carregar um valor pré-definido próximo ao limite máximo de representação do contador.

Internamente, o projeto utiliza um registrador de 33 bits (`counter_reg`), sendo os 32 bits menos significativos disponibilizados na saída `counter_out` e o bit mais significativo utilizado para gerar o sinal `counter_overflow`. Essa estrutura permite detectar quando a contagem ultrapassa a capacidade de representação de 32 bits.

O carregamento de um valor próximo ao limite superior do contador foi incluído para acelerar a ocorrência da condição de overflow durante a simulação, reduzindo o tempo necessário para validar essa funcionalidade.

8.2. Estímulos Aplicados

Inicialmente foi aplicado um reset assíncrono ativo em nível baixo para garantir a correta inicialização do contador. Após a liberação do reset, o sinal de habilitação (`en`) foi ativado, permitindo que o contador realizasse incrementos sucessivos sincronizados com o clock.

Em seguida, foi aplicado um estímulo de carregamento através do sinal `load`, responsável por inserir no contador um valor próximo ao limite máximo de contagem. Após o carregamento, a contagem permaneceu habilitada para verificar a

evolução dos valores armazenados, a ocorrência do overflow e o retorno da contagem para valores baixos após o estouro.

Na etapa final dos testes, o sinal de habilitação foi desativado para verificar se o contador mantinha o valor armazenado sem realizar novas contagens. Ao todo foram executadas 22 verificações, abrangendo as operações de reset, contagem, carregamento, overflow e retenção de estado.

Durante o desenvolvimento do mecanismo de self-checking foi necessário ajustar o modelo de referência para reproduzir exatamente o comportamento do RTL fornecido. Foi identificado que a constante utilizada na operação de carregamento possuía uma quantidade de bits diferente da indicada na declaração, fazendo com que o simulador realizasse truncamento automático do valor. O modelo de referência foi então adaptado para utilizar exatamente a mesma constante empregada no RTL, garantindo correspondência total entre os valores esperados e observados durante a simulação.

8.3. Resultados Obtidos

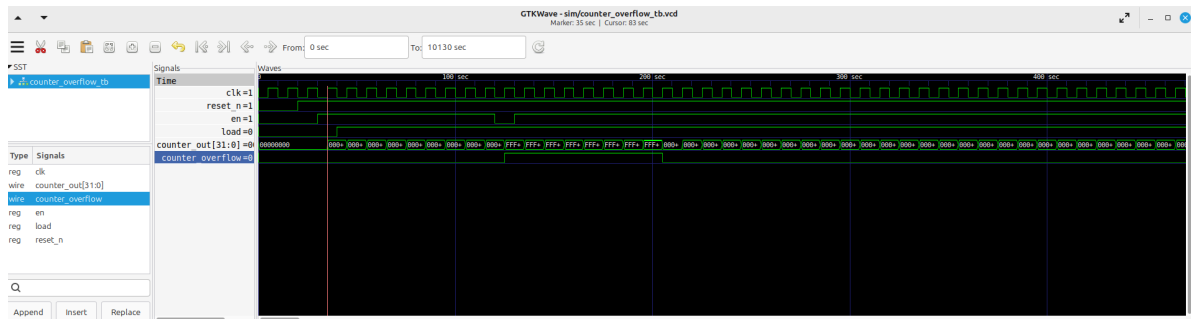
8.3.1. Testbench Original

O testbench original foi utilizado para validar o funcionamento do contador de 32 bits com detecção de overflow. Inicialmente, o circuito foi mantido em reset para garantir a inicialização correta do registrador interno. Após a liberação do reset, o sinal de habilitação (**en**) foi ativado, permitindo o início da contagem sincronizada com o clock.

Durante a simulação, o sinal **load** foi acionado para carregar um valor pré-definido próximo ao limite máximo representável em 32 bits. A partir desse valor, o contador continuou incrementando até que a condição de overflow fosse atingida. O comportamento observado confirmou que o circuito realiza corretamente tanto a operação de contagem quanto a detecção de estouro.

Também foi verificado o comportamento do sinal de habilitação. Quando **en** foi desativado, a contagem foi interrompida, permanecendo estável até que o sinal fosse novamente ativado. Após a reabilitação do contador, a contagem prosseguiu normalmente a partir do valor armazenado anteriormente.

Os resultados observados demonstram que o circuito executa corretamente as funções de reset, carregamento de valor inicial, contagem síncrona e sinalização de overflow.



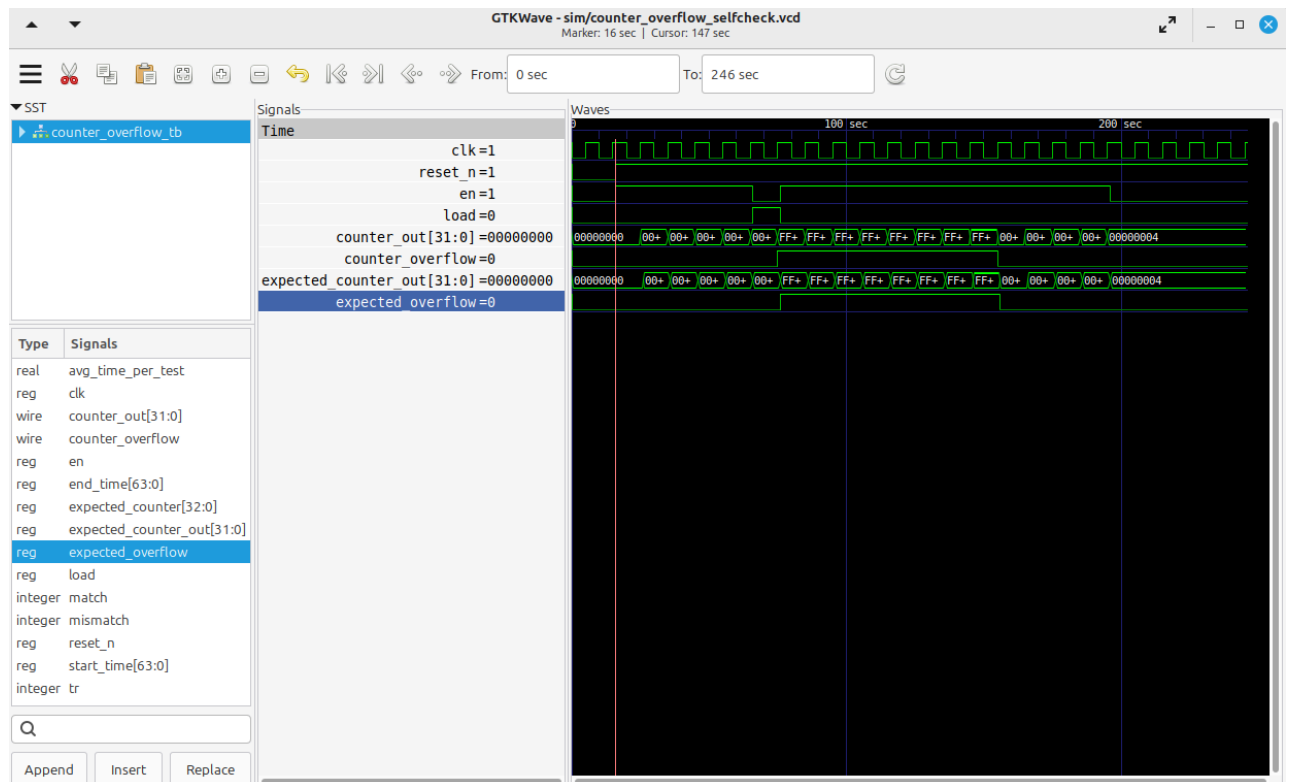
8.3.2. Testbench com Self-checking

Foi implementado um mecanismo de self-checking capaz de calcular automaticamente os valores esperados para as saídas `counter_out` e `counter_overflow`. Um modelo de referência reproduziu as operações realizadas pelo contador, permitindo comparar automaticamente os resultados produzidos pelo DUT com os valores calculados durante a execução dos testes.

O processo de verificação incluiu a validação da inicialização por reset, da operação de contagem, do carregamento de valores próximos ao limite superior e da detecção de overflow. Também foi verificado o comportamento do contador quando o sinal de habilitação foi desativado.

Ao final da simulação foram aplicados 22 estímulos de verificação, resultando em 22 aprovações e nenhuma falha. Os resultados demonstram que o comportamento observado no DUT corresponde exatamente ao comportamento previsto pelo modelo de referência.

Além da validação funcional, foi realizada uma análise temporal utilizando a função `$time` do SystemVerilog. O tempo total de execução registrado foi de 226 unidades de simulação, resultando em um tempo médio de aproximadamente 10,27 unidades por estímulo aplicado.



8.4. Análise das Waveforms

A waveform obtida a partir do testbench com self-checking demonstra a correta operação do contador em todas as etapas da simulação. Observa-se que os sinais **expected_counter_out** e **counter_out** permanecem coincidentes durante toda a execução dos testes, evidenciando que o comportamento do DUT corresponde exatamente ao previsto pelo modelo de referência.

Também é possível verificar a coincidência entre os sinais **expected_overflow** e **counter_overflow**, confirmando a correta geração da indicação de overflow quando o contador ultrapassa a capacidade de representação dos 32 bits disponíveis na saída.

Após a liberação do reset, a contagem é iniciada e os valores são incrementados sincronizadamente com as bordas de subida do clock. Em seguida, observa-se o carregamento do valor próximo ao limite máximo do contador por meio do sinal **load**. A partir desse ponto, a contagem prossegue até atingir a condição de overflow, quando ocorre a ativação do bit de overflow e a reinicialização dos bits menos significativos da saída.

Na etapa final da simulação, quando o sinal `en` é desativado, o valor armazenado permanece constante, demonstrando que o mecanismo de habilitação do contador opera corretamente. A coincidência total entre os sinais observados e esperados confirma a validade do modelo de referência utilizado no self-checking.

8.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do contador de 32 bits com detecção de overflow. Foram avaliadas todas as funcionalidades principais do circuito, incluindo reset, habilitação da contagem, carregamento de valores pré-definidos, detecção de overflow e manutenção do valor armazenado quando a contagem é interrompida.

A implementação do mecanismo de self-checking permitiu validar automaticamente os resultados produzidos pelo DUT, eliminando a necessidade de inspeção manual de cada transação. Durante a simulação foram realizadas 22 verificações, todas aprovadas, sem ocorrência de falhas.

A análise das waveforms confirmou a correspondência entre os sinais produzidos pelo circuito e os valores calculados pelo modelo de referência. Também foi possível observar corretamente o comportamento do overflow e a influência dos sinais de controle sobre a operação do contador.

Os resultados obtidos demonstram que o circuito atende às especificações funcionais propostas e que o ambiente de verificação desenvolvido foi capaz de validar tanto o comportamento lógico quanto os aspectos temporais do projeto.

9. Circuito 7 - Up/down Counter

9.1. Descrição do Circuito

O circuito analisado implementa um contador bidirecional composto por dois contadores de 4 bits operando simultaneamente. O primeiro contador, denominado `up_counter`, realiza contagem crescente, enquanto o segundo, denominado `down_counter`, realiza contagem decrescente. Ambos os contadores são sincronizados pelo mesmo sinal de clock e compartilham os sinais de habilitação e reset.

Quando o sinal de reset é acionado, o contador crescente é inicializado com o valor zero e o contador decrescente é inicializado com o valor máximo representável em 4 bits (1111). Após a liberação do reset e com o sinal de habilitação ativo, os contadores passam a atualizar seus valores a cada borda de subida do clock.

O contador crescente incrementa seu valor em uma unidade a cada ciclo, enquanto o contador decrescente reduz seu valor em uma unidade. Por se tratarem de contadores de 4 bits, ambos apresentam comportamento cíclico, retornando ao valor inicial após atingirem seus limites máximos ou mínimos.

9.2. Estímulos Aplicados

Os testes foram iniciados com a aplicação do sinal de reset para garantir a correta inicialização dos registradores internos do circuito. Após a verificação do estado inicial, o reset foi liberado e o sinal de habilitação foi ativado para permitir a execução das contagens crescente e decrescente.

Foi implementado um modelo de referência responsável por calcular os valores esperados para os sinais `up_counter` e `down_counter` ao longo da simulação. Devido às características específicas do circuito após a liberação do reset, foi realizada uma etapa de sincronização entre o modelo de referência e o DUT antes do início das verificações automáticas.

Durante a simulação foram aplicadas 21 verificações automáticas, cobrindo a evolução completa dos contadores ao longo de diversos ciclos de clock, incluindo a transição pelos limites máximos e mínimos representáveis em 4 bits. Além disso, foi realizada uma medição temporal utilizando a função `$time`, registrando o tempo total de execução e o tempo médio por transação.

9.3. Resultados Obtidos

9.3.1. Testbench Original

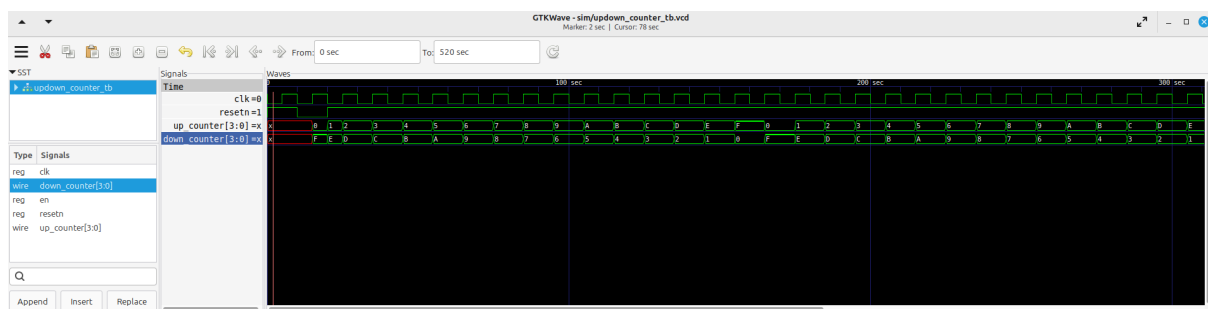
A simulação realizada com o testbench original permitiu verificar o funcionamento simultâneo dos dois contadores implementados no circuito. Após a aplicação do reset, observou-se que o contador crescente foi inicializado com o

valor zero, enquanto o contador decrescente assumiu o valor máximo representável em 4 bits.

Com o sinal de habilitação ativo, ambos os contadores passaram a ser atualizados a cada borda de subida do clock. O sinal `up_counter` apresentou uma sequência crescente de valores, enquanto o sinal `down_counter` apresentou uma sequência decrescente correspondente.

Também foi possível verificar o comportamento cíclico dos contadores. O contador crescente retornou para zero após atingir o valor hexadecimal `F`, enquanto o contador decrescente retornou para `F` após atingir o valor zero. Esse comportamento é compatível com a capacidade de representação de um contador de 4 bits.

Os resultados observados demonstram que o circuito executa corretamente as operações de inicialização, contagem crescente, contagem decrescente e recirculação dos valores ao atingir os limites da faixa de operação.



9.3.2. Testbench com Self-checking

Foi desenvolvido um testbench com mecanismo de self-checking capaz de comparar automaticamente os valores produzidos pelo circuito sob teste com aqueles calculados pelo modelo de referência. As verificações foram realizadas a cada ciclo de clock após a sincronização inicial entre o DUT e o modelo de referência.

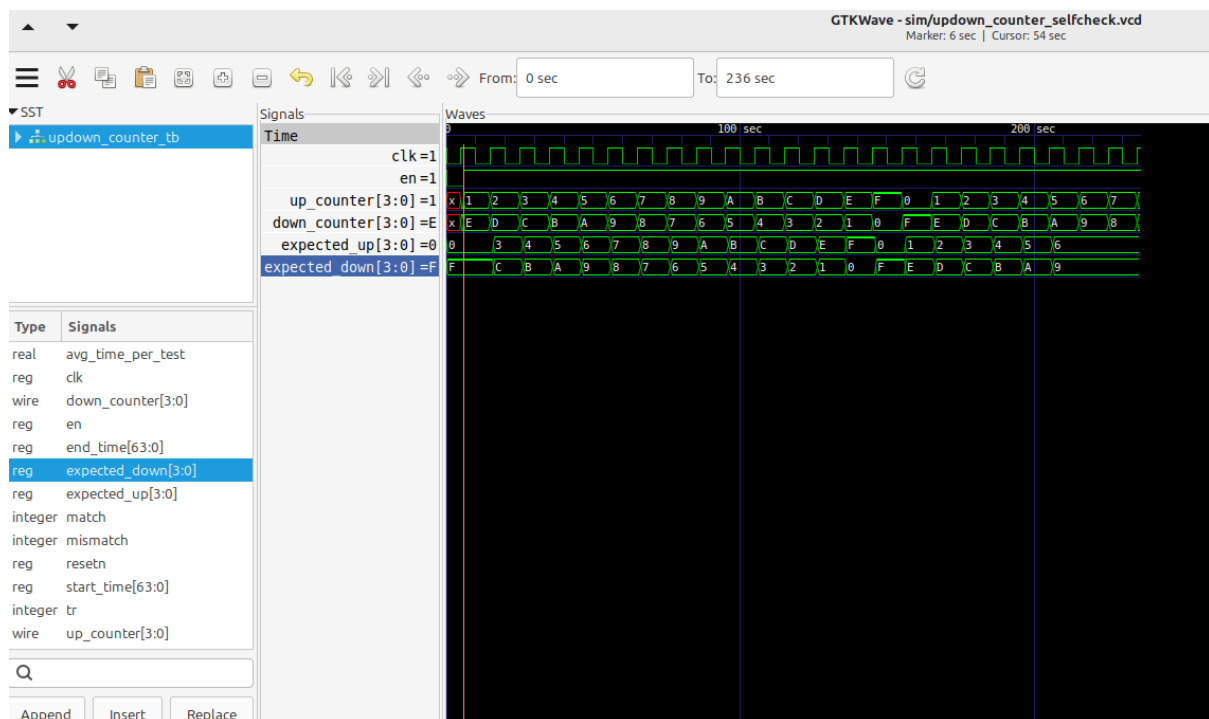
O relatório final da simulação registrou a aplicação de 21 estímulos, todos aprovados pelo mecanismo de comparação automática. Nenhuma divergência foi identificada entre os sinais produzidos pelo circuito e os valores esperados pelo modelo de referência.

Além da validação funcional, foi realizada uma análise temporal da simulação. O tempo total de execução registrado foi de 216 unidades de simulação,

resultando em um tempo médio de aproximadamente 10,29 unidades por transação verificada.

Os resultados obtidos demonstram que o circuito executa corretamente as operações de contagem crescente e decrescente previstas em sua especificação.

No início da simulação observa-se uma região em vermelho nos sinais `up_counter`, `down_counter`, `expected_up` e `expected_down`, indicando valores indefinidos (X). Esse comportamento ocorre porque os registradores do circuito e as variáveis utilizadas pelo modelo de referência ainda não foram completamente inicializados durante os primeiros instantes da simulação. Além disso, o testbench implementa uma etapa de sincronização entre o DUT (Device Under Test) e o modelo de referência logo após a liberação do reset. Durante esse período inicial, os sinais ainda não possuem valores válidos para comparação, motivo pelo qual o simulador os representa como indefinidos. Após a aplicação do reset e a sincronização dos sinais esperados com os valores produzidos pelo circuito, os sinais passam a assumir valores válidos e a região vermelha desaparece. A partir desse ponto, observa-se a correspondência entre `up_counter` e `expected_up`, bem como entre `down_counter` e `expected_down`, confirmando o correto funcionamento do mecanismo de self-checking.



9.4. Análise das Waveforms

A análise da waveform confirma o correto funcionamento do mecanismo de self-checking implementado para o contador bidirecional. Observa-se que os sinais `expected_up` e `expected_down`, produzidos pelo modelo de referência, permanecem coincidentes com os sinais `up_counter` e `down_counter` gerados pelo DUT durante toda a simulação.

Após a etapa inicial de sincronização, o contador crescente passa a incrementar seus valores de forma sequencial, enquanto o contador decrescente realiza decrementos sucessivos a cada borda de subida do clock. As transições observadas nas saídas ocorrem de maneira sincronizada, evidenciando o comportamento esperado para um circuito sequencial controlado por clock.

Também é possível observar o comportamento cíclico dos contadores. O sinal `up_counter` retorna ao valor zero após atingir o valor hexadecimal `F`, enquanto o sinal `down_counter` retorna para `F` após atingir zero. Esse comportamento demonstra o correto tratamento das condições de overflow e underflow inerentes à representação de 4 bits.

A coincidência visual entre os sinais esperados e observados ao longo de toda a simulação confirma a consistência do modelo de referência e a correta implementação do circuito.

9.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do contador bidirecional de 4 bits. Foram verificadas as operações de inicialização por reset, contagem crescente, contagem decrescente e comportamento cíclico dos contadores ao atingirem os limites da faixa de representação.

A utilização de um testbench com mecanismo de self-checking permitiu automatizar completamente o processo de validação, comparando as saídas do DUT com os valores calculados pelo modelo de referência. Após a sincronização inicial necessária para alinhar o comportamento do modelo ao circuito, todas as verificações realizadas foram aprovadas.

A análise das waveforms demonstrou total correspondência entre os sinais observados e os valores esperados durante toda a execução dos testes. O relatório

final registrou 21 verificações aprovadas, nenhuma falha e um tempo médio de aproximadamente 10,29 unidades de simulação por transação.

Dessa forma, os resultados obtidos demonstram que o circuito atende às especificações funcionais propostas e que o ambiente de verificação desenvolvido foi capaz de validar adequadamente seu comportamento lógico e temporal.

10. Circuito 8 - Adder 32

10.1. Descrição do Circuito

O circuito implementa um somador síncrono de 32 bits responsável por realizar a soma de dois operandos de entrada (**A** e **B**). Internamente, a soma é armazenada em um registrador de 33 bits, permitindo representar os 32 bits do resultado e um bit adicional utilizado para indicar a ocorrência de carry ou overflow da operação.

A atualização das saídas ocorre nas bordas de subida do clock quando o sinal de habilitação (**en**) está ativo. O sinal de reset assíncrono ativo em nível baixo (**reset_n**) inicializa o registrador interno, forçando o resultado da soma e o sinal de carry para zero. O resultado da operação é disponibilizado na saída **sum[31:0]**, enquanto o bit excedente da soma é disponibilizado pela saída **carry_out**.

10.2. Estímulos Aplicados

Os testes foram iniciados com a aplicação do reset para garantir a correta inicialização dos registradores internos do circuito. Após a liberação do reset e a ativação do sinal de habilitação, foram aplicados quatro pares de operandos de 32 bits, correspondentes aos mesmos vetores utilizados no testbench original.

Os estímulos foram selecionados de forma a verificar tanto operações que permanecem dentro da faixa de representação de 32 bits quanto situações em que ocorre geração de carry. Para cada conjunto de operandos, o modelo de referência calculou automaticamente o resultado esperado da soma e o valor esperado para o sinal de carry.

Além da validação funcional, foi implementada uma medição temporal utilizando a função **\$time** do SystemVerilog. O tempo de execução foi registrado no início e ao final dos testes, permitindo calcular a duração total da simulação e o tempo médio gasto por transação verificada.

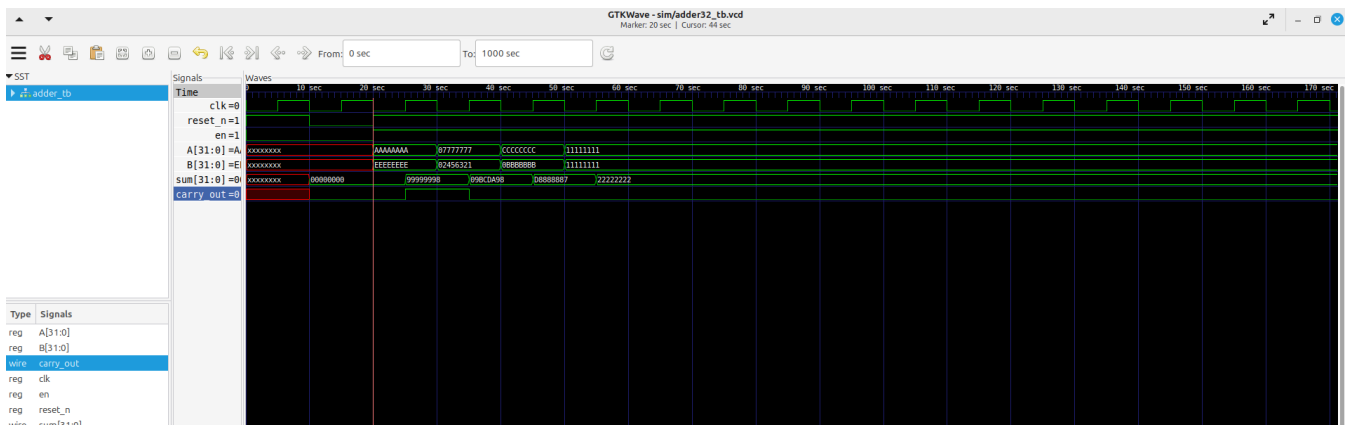
10.3. Resultados Obtidos

10.3.1. Testbench Original

No testbench original foram aplicados quatro conjuntos de operandos ao circuito, permitindo observar o comportamento da soma em diferentes condições de operação. Após a liberação do reset, os valores foram aplicados sequencialmente às entradas **A** e **B**, sendo os resultados observados através da waveform gerada pela simulação.

As operações realizadas produziram resultados compatíveis com os valores esperados para cada par de operandos. Também foi possível verificar a ativação do sinal **carry_out** durante a soma dos operandos **AAAAAAAh** e **EEEEEEEHh**, demonstrando a correta detecção de overflow além dos 32 bits disponíveis na saída principal.

Os resultados observados confirmaram o correto funcionamento funcional do somador antes da implementação do mecanismo de self-checking.

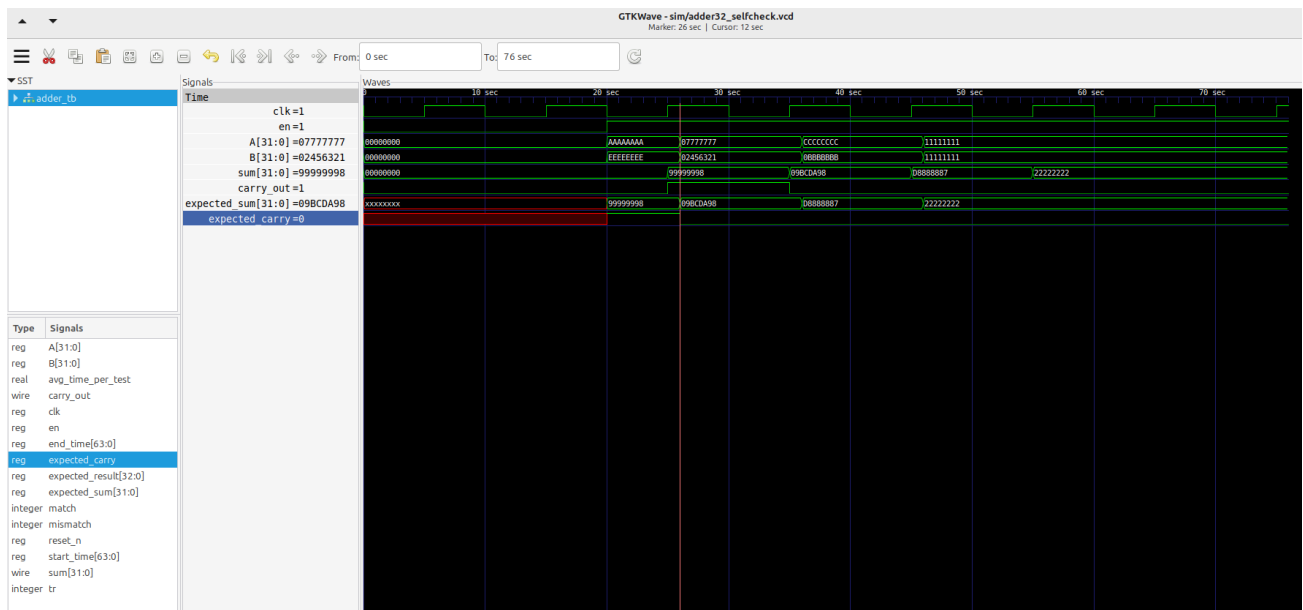


10.3.2. Testbench com Self-checking

Foi desenvolvido um ambiente de verificação baseado em self-checking, no qual um modelo de referência calcula automaticamente os resultados esperados para a soma e para o sinal de carry. Após cada estímulo aplicado, os valores produzidos pelo DUT são comparados automaticamente com os resultados calculados pelo modelo de referência.

Durante a execução da simulação foram aplicados quatro estímulos de teste. Todas as comparações realizadas apresentaram correspondência entre os valores observados e os valores esperados, resultando em quatro verificações aprovadas e nenhuma falha.

O relatório final da simulação registrou um tempo total de execução de 56 unidades de simulação, resultando em um tempo médio de aproximadamente 14 unidades por transação verificada.



10.4. Análise das Waveforms

A waveform obtida a partir do testbench com self-checking demonstra a correta operação do somador e do mecanismo de verificação automática. Inicialmente observa-se uma região em vermelho nos sinais **expected_sum** e **expected_carry**, indicando valores indefinidos (X). Esse comportamento é esperado, pois o modelo de referência ainda não havia realizado o cálculo do primeiro resultado esperado durante os instantes iniciais da simulação.

Após a aplicação do primeiro estímulo, os sinais esperados passam a assumir valores válidos e permanecem sincronizados com as saídas produzidas pelo DUT. Observa-se que os valores de **expected_sum** coincidem com os valores presentes em **sum**, enquanto **expected_carry** acompanha corretamente o comportamento de **carry_out**.

A primeira operação produz um carry de saída devido ao resultado exceder a capacidade de representação de 32 bits. Nas demais operações, os resultados permanecem dentro da faixa representável, mantendo o sinal de carry desativado. A correspondência entre os sinais esperados e observados durante toda a simulação confirma o correto funcionamento do somador e do ambiente de verificação implementado.

10.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do somador síncrono de 32 bits. Foram verificadas operações de soma envolvendo diferentes combinações de operandos, incluindo casos com geração de carry e casos sem ocorrência de overflow.

A implementação do mecanismo de self-checking permitiu automatizar completamente o processo de validação, comparando as saídas produzidas pelo DUT com os resultados calculados por um modelo de referência. Durante a simulação foram executadas quatro verificações, todas aprovadas, sem ocorrência de falhas.

A análise das waveforms demonstrou total correspondência entre os sinais observados e os valores esperados, evidenciando o correto funcionamento do circuito e do ambiente de verificação. Além disso, a medição temporal registrou um tempo total de execução de 56 unidades de simulação, resultando em um tempo médio de aproximadamente 14 unidades por transação.

Dessa forma, os resultados obtidos demonstram que o circuito atende às especificações funcionais propostas e que o ambiente de verificação desenvolvido foi capaz de validar adequadamente seu comportamento lógico e temporal.

11. Circuito 9 - Multiplier 16x16

11.1. Descrição do Circuito

O circuito implementa um multiplicador síncrono de 16 bits responsável por realizar a multiplicação entre dois operandos de entrada (`op_a` e `op_b`). O resultado da operação é armazenado em um registrador de 32 bits e disponibilizado na saída `multi_out`.

A atualização da saída ocorre na borda de subida do clock quando o sinal de habilitação (**en**) está ativo. O circuito também possui um sinal de reset assíncrono ativo em nível baixo (**reset_n**), utilizado para inicializar o registrador interno com zero. Como o produto de dois operandos de 16 bits pode ocupar até 32 bits, a largura da saída é suficiente para representar integralmente o resultado da multiplicação sem necessidade de sinalização adicional de overflow.

11.2. Estímulos Aplicados

Inicialmente foi aplicado o sinal de reset para garantir a correta inicialização do circuito. Após a liberação do reset, o sinal de habilitação foi ativado, permitindo a execução das operações de multiplicação.

No testbench original foram utilizados dois pares de operandos fornecidos pelo professor. Já no testbench com self-checking foram adicionados casos extras de teste para aumentar a cobertura funcional do circuito. Os estímulos utilizados incluíram operações com valores elevados, multiplicação por zero, multiplicação por um, multiplicação de valores intermediários e o caso máximo representável em 16 bits (**FFFFh × FFFFh**).

Foi implementado um modelo de referência responsável por calcular automaticamente o resultado esperado através da operação matemática de multiplicação. Além disso, foram registradas métricas de desempenho utilizando a função **\$time**, permitindo calcular o tempo total de execução da simulação e o tempo médio gasto por estímulo aplicado.

11.3. Resultados Obtidos

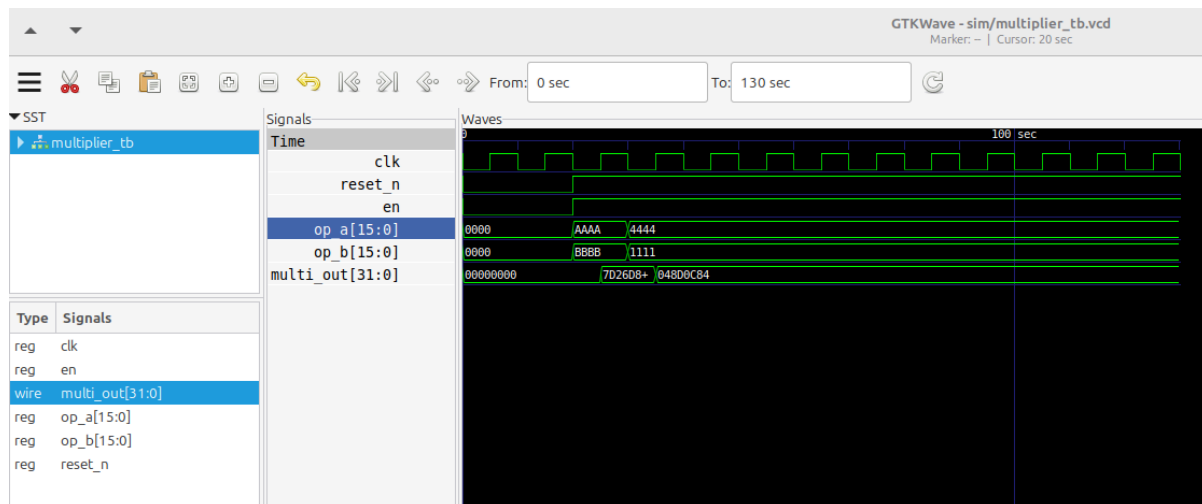
11.3.1. Testbench Original

O testbench original aplicou dois conjuntos de operandos ao multiplicador. Após a liberação do reset, os valores foram apresentados nas entradas **op_a** e **op_b**, sendo os produtos observados diretamente na saída **multi_out** através da waveform.

Os resultados obtidos corresponderam aos valores esperados para as operações realizadas. A multiplicação entre **AAAAh** e **BBBBh** produziu o resultado

7D26D82Eh, enquanto a multiplicação entre 4444h e 1111h resultou em 048D0C84h.

A análise das formas de onda demonstrou que a saída foi atualizada corretamente após as bordas de subida do clock, confirmando o funcionamento síncrono do circuito.



11.3.2. Testbench com Self-checking

Foi desenvolvido um ambiente de verificação baseado em self-checking, no qual um modelo de referência calcula automaticamente o produto esperado para cada conjunto de operandos aplicado ao DUT. Após cada operação, o valor observado em `multi_out` é comparado automaticamente com o resultado calculado pelo modelo de referência.

Durante a simulação foram executados seis estímulos distintos. Todas as verificações apresentaram correspondência entre os valores observados e os valores esperados, resultando em seis testes aprovados e nenhuma falha.

O relatório final registrou os seguintes resultados:

Stimuli Applied : 6

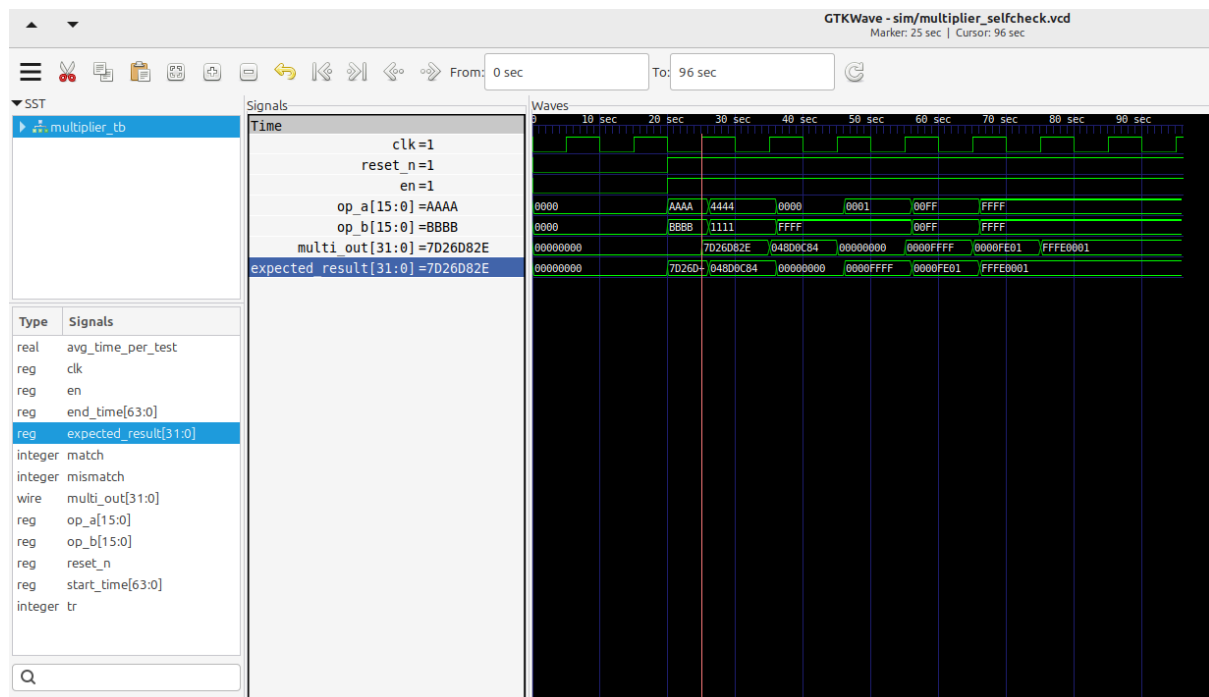
PASS : 6

FAIL : 0

Execution Time : 76

Average Time/Test : 12.67

Os resultados demonstram que o multiplicador executou corretamente todas as operações verificadas.



11.4. Análise das Waveforms

A waveform obtida com o testbench de self-checking evidencia o correto funcionamento tanto do circuito quanto do mecanismo de verificação automática. Observa-se que os sinais **expected_result** e **multi_out** apresentam exatamente os mesmos valores durante toda a simulação, indicando total concordância entre o modelo de referência e o DUT.

Após a liberação do reset, os operandos são aplicados sequencialmente às entradas. Para cada conjunto de valores, o resultado esperado calculado pelo modelo de referência aparece simultaneamente ao resultado produzido pelo multiplicador, demonstrando que a saída do circuito está correta.

Os casos de teste permitiram validar diferentes condições de operação. A multiplicação por zero produziu saída nula, a multiplicação por um preservou o valor do operando, os valores intermediários geraram produtos compatíveis com os cálculos esperados e o caso máximo ($FFFFh \times FFFFh$) resultou em **FFFE0001h**, comprovando a correta utilização dos 32 bits da saída.

A ausência de divergências entre `expected_result` e `multi_out` durante toda a simulação confirma a consistência do modelo de referência e a correta implementação do multiplicador.

11.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do multiplicador síncrono de 16×16 bits. Foram verificadas operações envolvendo diferentes combinações de operandos, incluindo multiplicações por zero, por um, valores intermediários e valores máximos representáveis.

A implementação do mecanismo de self-checking permitiu automatizar completamente o processo de validação, eliminando a necessidade de inspeção manual dos resultados. Todas as comparações realizadas entre o DUT e o modelo de referência foram aprovadas, resultando em 100% de sucesso nos testes executados.

A análise das waveforms demonstrou total correspondência entre os sinais esperados e observados, confirmando que o circuito produz corretamente os produtos das operações de multiplicação e atualiza sua saída de forma sincronizada com o clock.

Dessa forma, os resultados obtidos demonstram que o circuito atende integralmente às especificações propostas e que o ambiente de verificação desenvolvido foi capaz de validar adequadamente seu comportamento funcional e temporal.

12. Circuito 10 - Multiplicação de Matriz

12.1. Descrição do Circuito

O circuito implementa a multiplicação entre duas matrizes 2×2 , cujos elementos são representados por valores de 8 bits. As matrizes de entrada são armazenadas nos sinais `A` e `B`, ambos com 32 bits, onde cada grupo de 8 bits representa um elemento da matriz. Internamente, as entradas são convertidas para uma estrutura matricial bidimensional, permitindo a realização das operações de multiplicação e soma necessárias para o cálculo do produto matricial.

A operação é executada na borda de subida do clock quando o sinal de habilitação (**en**) está ativo. O resultado é armazenado em uma matriz interna e posteriormente concatenado para formar a saída **Res[31:0]**. O circuito também possui um sinal de reset assíncrono ativo em nível baixo (**rstn**), utilizado para inicializar a matriz de saída.

12.2. Estímulos Aplicados

Inicialmente foi aplicado o sinal de reset para garantir a correta inicialização dos registradores internos do circuito. Após a liberação do reset e a ativação do sinal de habilitação, foram aplicados diferentes pares de matrizes de entrada para verificar o funcionamento da multiplicação matricial.

No testbench original foram utilizados dois conjuntos básicos de matrizes compostas por elementos iguais a 1 e iguais a 2. No testbench com self-checking foram adicionados casos adicionais para ampliar a cobertura funcional do circuito, incluindo matrizes nulas, matrizes com valores distintos e matrizes contendo elementos máximos de 8 bits.

Para cada estímulo aplicado, um modelo de referência calculou automaticamente o resultado esperado da multiplicação matricial utilizando as mesmas equações implementadas no DUT. Também foi realizada a medição do tempo de execução da simulação por meio da função **\$time**, permitindo calcular o tempo total de execução e o tempo médio por estímulo.

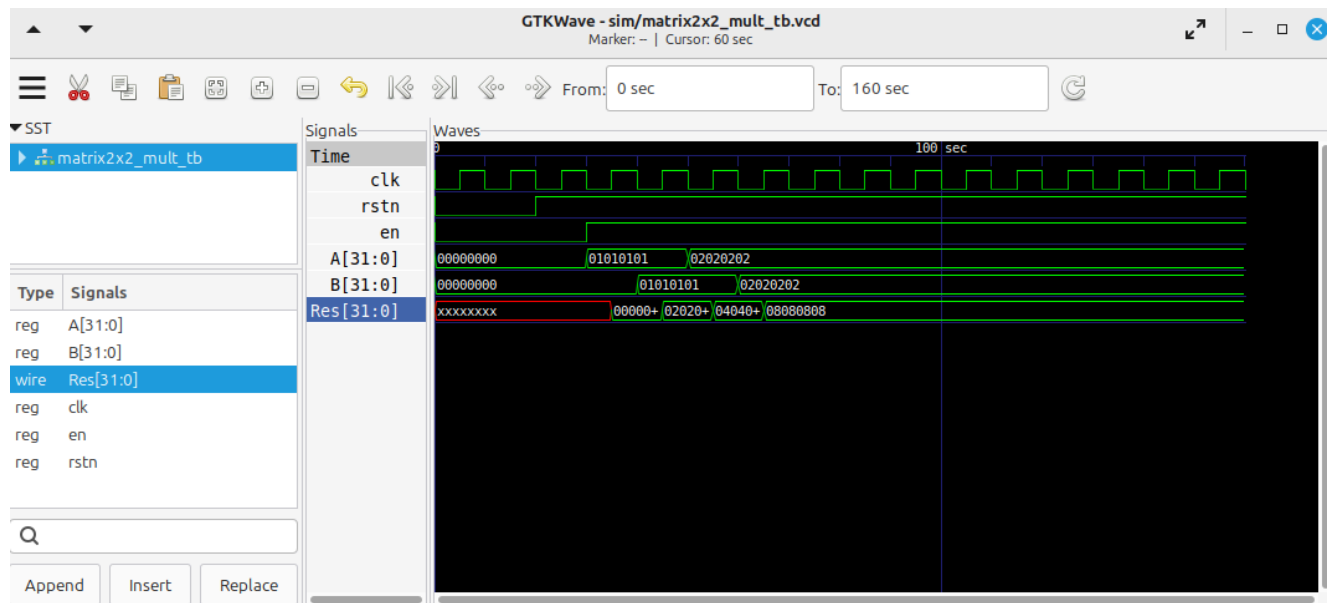
12.3. Resultados Obtidos

12.3.1. Testbench Original

O testbench original permitiu verificar o comportamento funcional básico do circuito. As primeiras matrizes aplicadas continham apenas elementos iguais a 1, resultando em uma matriz de saída composta por valores iguais a 2, representada por **02020202h**.

Em seguida, foi aplicada uma combinação intermediária na qual uma das matrizes possuía elementos iguais a 2 enquanto a outra permanecia com elementos iguais a 1. O resultado observado foi **04040404h**, compatível com a multiplicação matricial esperada.

Por fim, ambas as matrizes passaram a conter elementos iguais a 2, produzindo como resultado **08080808h**. Todos os resultados observados foram compatíveis com os cálculos teóricos da multiplicação de matrizes 2×2 .



12.3.2. Testbench com Self-checking

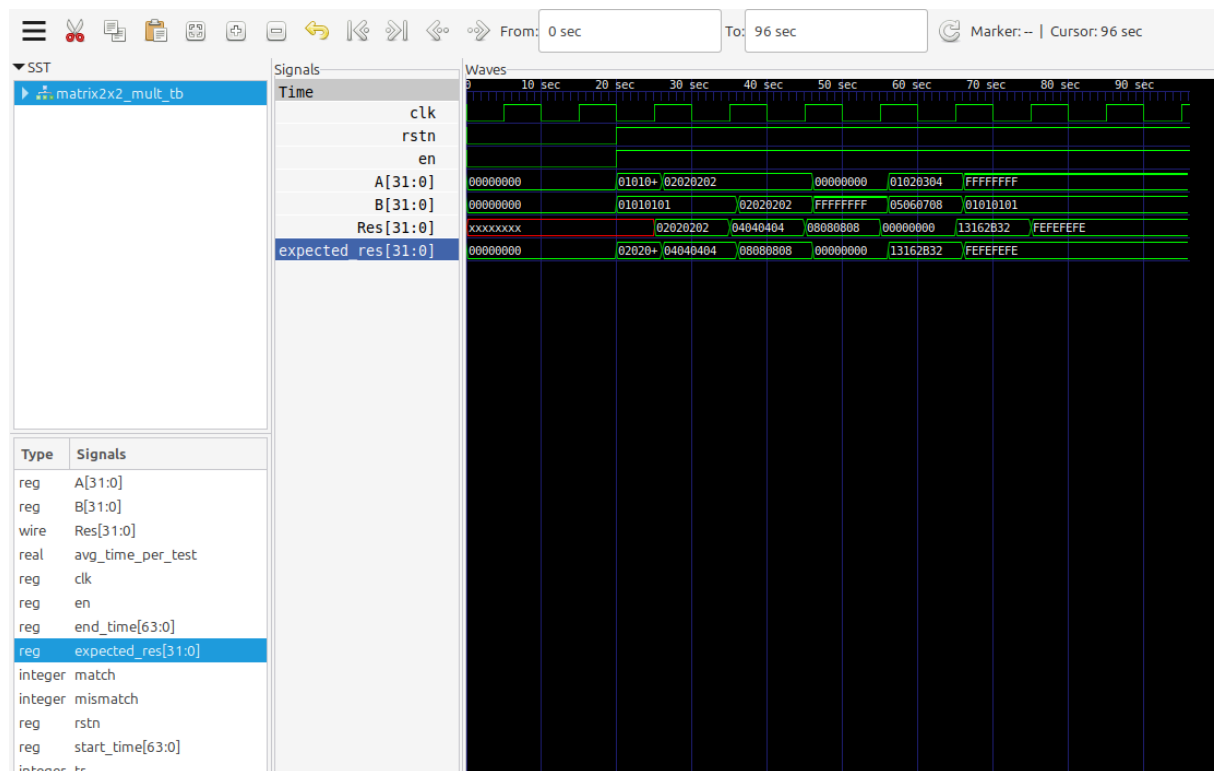
Foi implementado um ambiente de verificação baseado em self-checking capaz de calcular automaticamente os resultados esperados para cada operação matricial. O modelo de referência extrai os elementos das matrizes de entrada, realiza as multiplicações e somas correspondentes e compara o resultado obtido com a saída produzida pelo DUT.

Durante a simulação foram aplicados seis conjuntos de estímulos distintos. Todas as verificações apresentaram correspondência entre os valores observados e os valores esperados, resultando em seis testes aprovados e nenhuma falha.

O relatório final apresentou os seguintes resultados:

```
Stimuli Applied      : 6
PASS                 : 6
FAIL                 : 0
Execution Time       : 76
Average Time/Test    : 12.67
```

Esses resultados demonstram que o circuito executou corretamente todas as operações matriciais verificadas.



12.4. Análise das Waveforms

A waveform obtida no testbench com self-checking demonstra o correto funcionamento tanto do circuito quanto do mecanismo de verificação automática. Inicialmente observa-se uma região em vermelho no sinal **Res**, correspondente aos valores indefinidos (X) presentes antes da inicialização completa do circuito. Esse comportamento é esperado em simulações digitais durante os instantes iniciais da execução.

Após a liberação do reset e a ativação do sinal de habilitação, os pares de matrizes são aplicados sequencialmente às entradas **A** e **B**. Para cada conjunto de estímulos, o sinal **expected_res** apresenta o resultado calculado pelo modelo de referência, enquanto o sinal **Res** apresenta o resultado produzido pelo DUT. Observa-se que ambos os sinais permanecem coincidentes durante toda a simulação. Os resultados **02020202h**, **04040404h**, **08080808h**, **00000000h**, **13162B32h** e **FEFEFEFEh** aparecem simultaneamente em **Res** e **expected_res**,

evidenciando que o circuito executa corretamente as operações de multiplicação matricial.

A coincidência entre os resultados observados e os valores calculados pelo modelo de referência confirma a correta implementação do algoritmo de multiplicação de matrizes e a eficácia do ambiente de verificação desenvolvido.

Observa-se no início da waveform uma região destacada em vermelho no sinal `Res[31:0]`, representando o estado lógico indefinido (X). Esse comportamento é normal em simulações digitais e ocorre porque, nos primeiros instantes da execução, o registrador responsável por armazenar o resultado da multiplicação ainda não recebeu um valor válido proveniente do circuito. Embora o sinal de reset (`rstn`) seja aplicado no início da simulação, o resultado somente passa a assumir valores definidos após a ocorrência da primeira borda de subida do clock com as condições adequadas de reset e habilitação. Após a inicialização do circuito, o sinal `Res` deixa o estado indefinido e passa a apresentar valores válidos, coincidindo perfeitamente com o sinal `expected_res` calculado pelo modelo de referência. Dessa forma, a presença dos valores X no início da simulação não caracteriza falha funcional, mas sim um comportamento esperado decorrente da fase de inicialização do hardware simulado.

12.5. Conclusão

Os testes realizados confirmaram o correto funcionamento do circuito de multiplicação de matrizes 2×2 . Foram verificadas diferentes combinações de matrizes de entrada, incluindo casos simples, casos intermediários, matrizes nulas e matrizes contendo valores elevados.

A implementação do mecanismo de self-checking permitiu automatizar completamente a validação do circuito, comparando os resultados produzidos pelo DUT com os valores calculados por um modelo de referência independente. Todas as verificações executadas foram aprovadas, resultando em 100% de sucesso nos testes realizados.

A análise das waveforms demonstrou total correspondência entre os sinais esperados e observados ao longo de toda a simulação, comprovando a correta execução das operações matriciais e a estabilidade do circuito. Além disso, as

métricas temporais obtidas indicaram uma execução eficiente do ambiente de verificação.

Dessa forma, conclui-se que o circuito atende integralmente às especificações propostas e que o ambiente de verificação desenvolvido foi capaz de validar adequadamente seu comportamento funcional e temporal.

13. Conclusão Geral

A realização desta atividade permitiu aplicar conceitos fundamentais de verificação funcional de sistemas digitais utilizando Verilog HDL e SystemVerilog. Foram analisados diversos circuitos digitais, incluindo multiplexadores, demultiplexadores, codificadores, decodificadores, árbitros, contadores, somadores, multiplicadores e multiplicadores matriciais, possibilitando a avaliação de diferentes tipos de arquiteturas combinacionais e sequenciais.

Inicialmente, os circuitos foram simulados utilizando os testbenches fornecidos, permitindo observar seu comportamento funcional por meio das waveforms geradas. Em seguida, os testbenches foram aprimorados com a implementação de mecanismos de self-checking, incorporando modelos de referência capazes de calcular automaticamente os resultados esperados e compará-los com as saídas produzidas pelos circuitos sob teste (DUTs). Essa abordagem tornou o processo de verificação mais robusto, automatizado e confiável, reduzindo a dependência de inspeções visuais manuais.

Durante o desenvolvimento dos testes, também foram identificadas e corrigidas situações relacionadas à sincronização entre os modelos de referência e os circuitos analisados, bem como particularidades de implementação presentes em alguns projetos fornecidos. Essas correções permitiram obter correspondência total entre os resultados observados e os resultados esperados, garantindo a validade dos mecanismos de verificação desenvolvidos.

Os resultados obtidos demonstraram que todos os circuitos avaliados atenderam às suas especificações funcionais. Em todos os testbenches com self-checking, os valores produzidos pelos DUTs coincidiram com aqueles calculados pelos modelos de referência, resultando em aprovação integral dos testes executados e ausência de falhas nas verificações finais.

A análise das waveforms complementou o processo de validação, permitindo observar aspectos importantes relacionados à inicialização dos sinais, atuação dos resets, comportamento síncrono em relação ao clock, propagação dos estímulos e atualização das saídas. Também foi possível verificar corretamente condições específicas, como overflow em contadores, arbitragem baseada em prioridade, operações aritméticas e multiplicações matriciais.

Por fim, a utilização da infraestrutura composta por Makefile, Icarus Verilog, VVP e GTKWave mostrou-se eficiente para automatizar as etapas de compilação, simulação e análise dos resultados. Dessa forma, os objetivos propostos para a atividade foram plenamente alcançados, evidenciando a importância das técnicas de verificação funcional e do uso de testbenches com self-checking no desenvolvimento de sistemas digitais confiáveis.